InstaGAN: Instance-aware Image-to-Image Translation
(Presented at ICLR 2019)

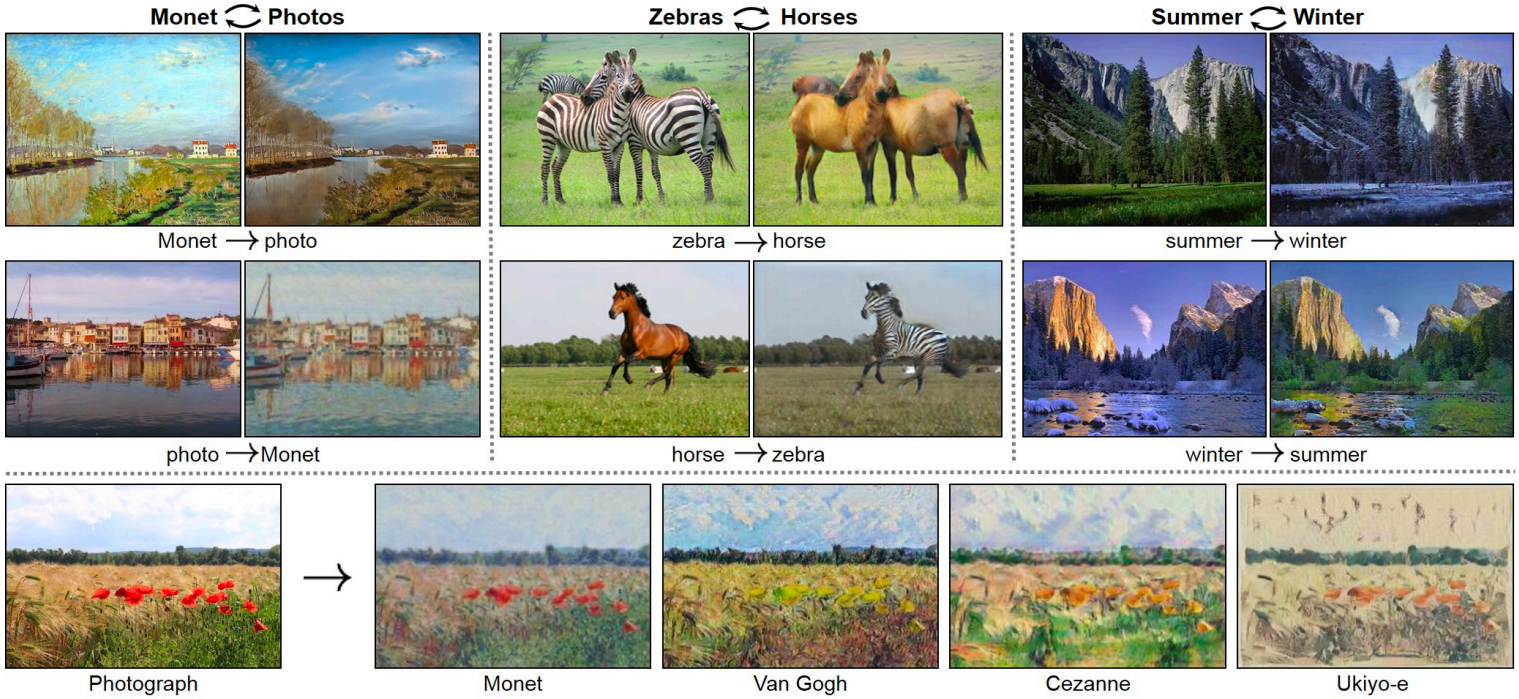
Sangwoo Mo[1], Minsu Cho[2], Jinwoo Shin[1]

[1]KAIST, [2]POSTECH

Korean Conference on Computer Vision, July 2019

Introduction: Image-to-Image Translation

- **Goal:** Transform target scene elements while preserving the original content



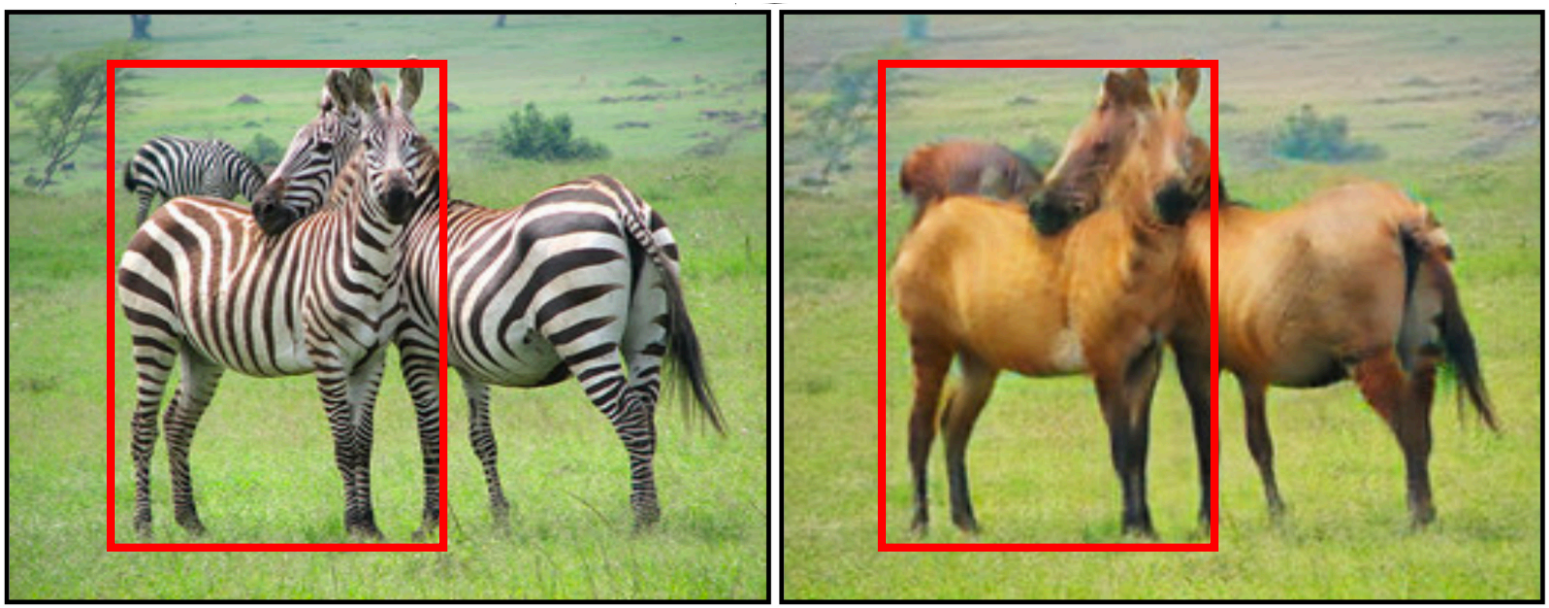
- Gained lots of attention due to the recent progress based on deep learning

Pix2pix (CVPR 2017), **2530** citations
CycleGAN (ICCV 2017), **2087** citations
DiscoGAN (ICML 2017), **493** citations
UNIT (NIPS 2017), **512** citations
StarGAN (CVPR 2018), **307** citations

* Image from CycleGAN (ICCV 2017)

Introduction: Image-to-Image Translation

- Training deep models for image-to-image translation requires two losses:
  1. **Domain loss:** Change the domain of an image (e.g., zebra→horse)
  2. **Content loss:** Keep the content of the original image (e.g., zebra's shape)



zebra **horse**

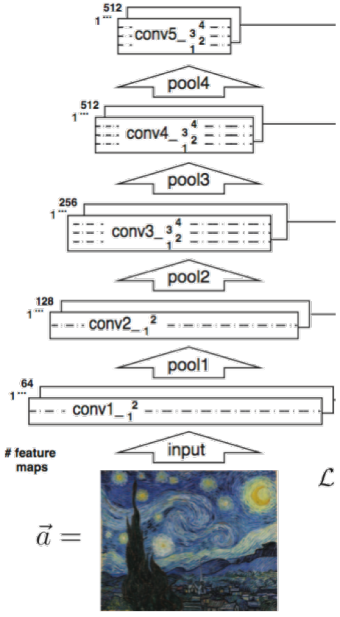
* Image from CycleGAN (ICCV 2017)

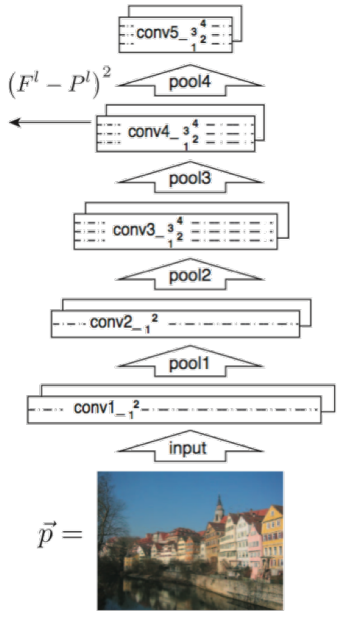
Introduction: Prior Work & Motivation

- **Prior work: Neural Style Transfer**
  - Solve an optimization problem $\hat{x} = \min_x \mathcal{L}_{\text{content}}(x_c, x) + \lambda \mathcal{L}_{\text{style}}(x_s, x)$

    (Given inputs x_c and x_s, generate $\hat{x}$ with content of x_c and style of x_s)
  - Both content and domain (style) loss are given by the pretrained network

    (e.g., ImageNet-pretrained VGGNet)





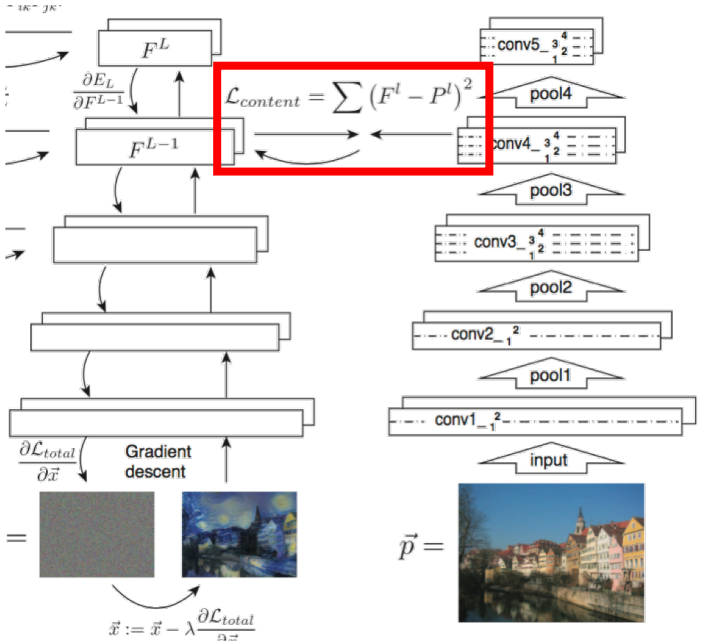
* Image from Neural Style Transfer (CVPR 2016)

Introduction: Prior Work & Motivation

- **Prior work: Neural Style Transfer**
  - **Content loss =** Match the features of high layers

$$\mathcal{L}_{\text{content}}(x_c, x) = \frac{1}{2}\sum_{i,k}(F^l_{ik}(x_c) - F^l_{ik}(x))^2$$

where F^l_{ik} is the feature of i-th location, k-th channel, and l-th layer



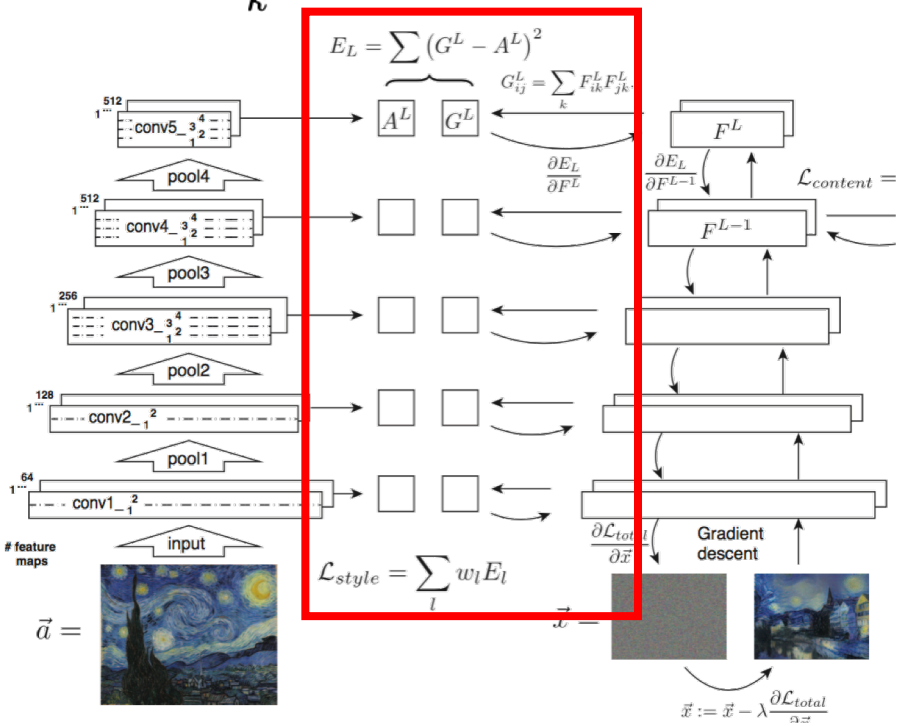
* Image from Neural Style Transfer (CVPR 2016)

Introduction: Prior Work & Motivation

- **Prior work: Neural Style Transfer**
  - **Domain (style) loss =** Match the feature statistics of layers

$$\mathcal{L}_{\text{style}}(x_s, x) = \sum_{i,j} (G^l_{ij}(x_s) - G^l_{ij}(x))^2$$

where $G^l_{ij} = \sum_k F^l_{ik} F^l_{jk}$ is the Gram matrix of the features

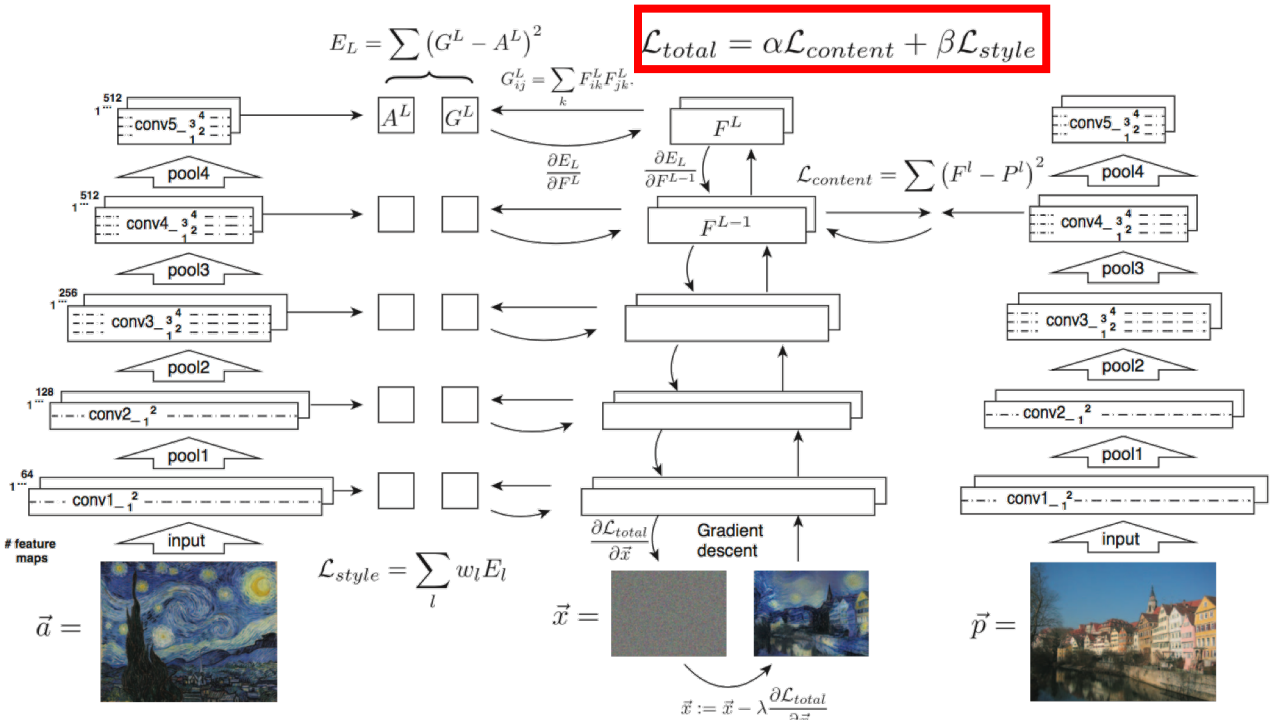


* Image from Neural Style Transfer (CVPR 2016)

Introduction: Prior Work & Motivation

- **Prior work: Neural Style Transfer**
  - Solve an optimization problem $\hat{x} = \min_x \mathcal{L}_{\text{content}}(x_c, x) + \lambda \mathcal{L}_{\text{style}}(x_s, x)$

    (Given inputs x_c and x_s, generate $\hat{x}$ with content of x_c and style of x_s)

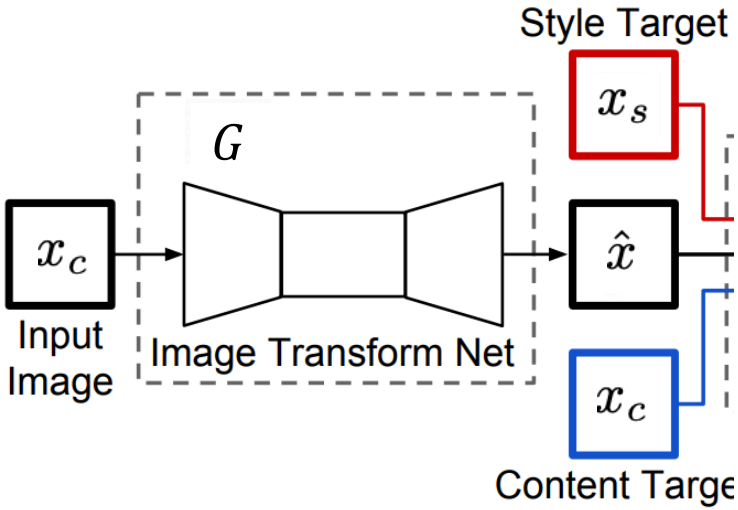


* Image from Neural Style Transfer (CVPR 2016)

Introduction: Prior Work & Motivation

- **Prior work: Fast Neural Style**
  - Instead an optimization, train a generator G that translates the style

    (Given input x_c, generate $\hat{x} \coloneqq G_s(x_c)$ with content of x_c and style of G_s)
  - It is 100x faster than Neural Style (solve a optimization for an inference)



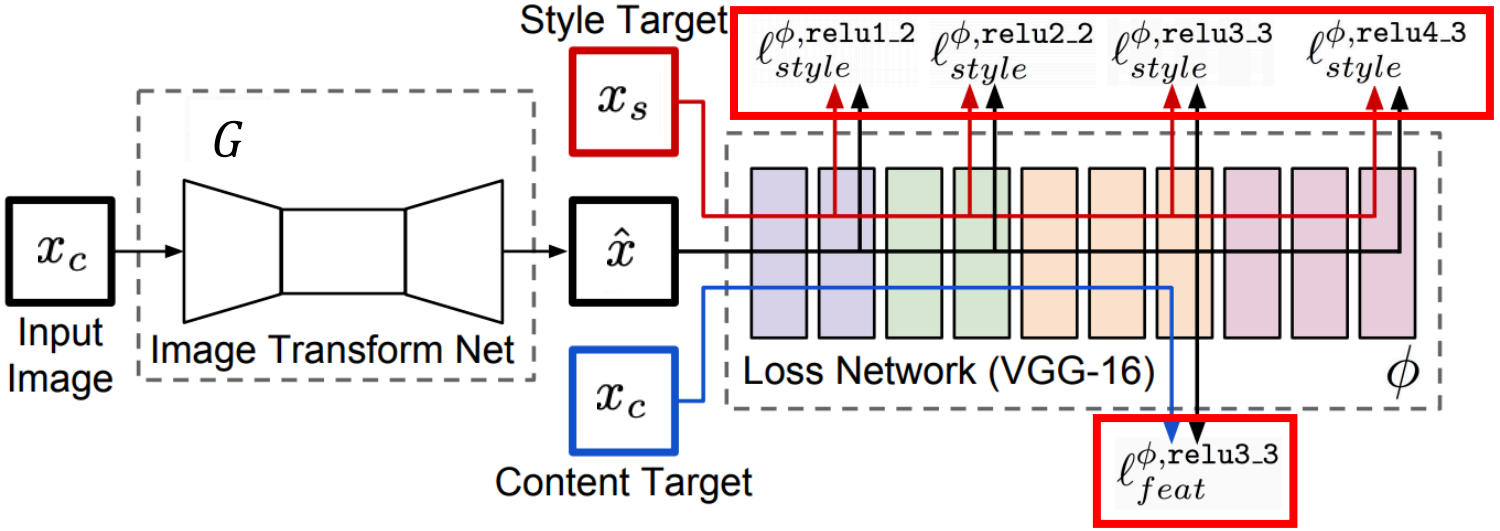
* Image from Fast Neural Style (ECCV 2016)

Introduction: Prior Work & Motivation

- **Prior work: Fast Neural Style**
  - Instead an optimization, train a generator G that translates the style

    (Given input x_c, generate $\hat{x} \coloneqq G_s(x_c)$ with content of x_c and style of G_s)
  - The network is trained to optimize the Neural Style objective

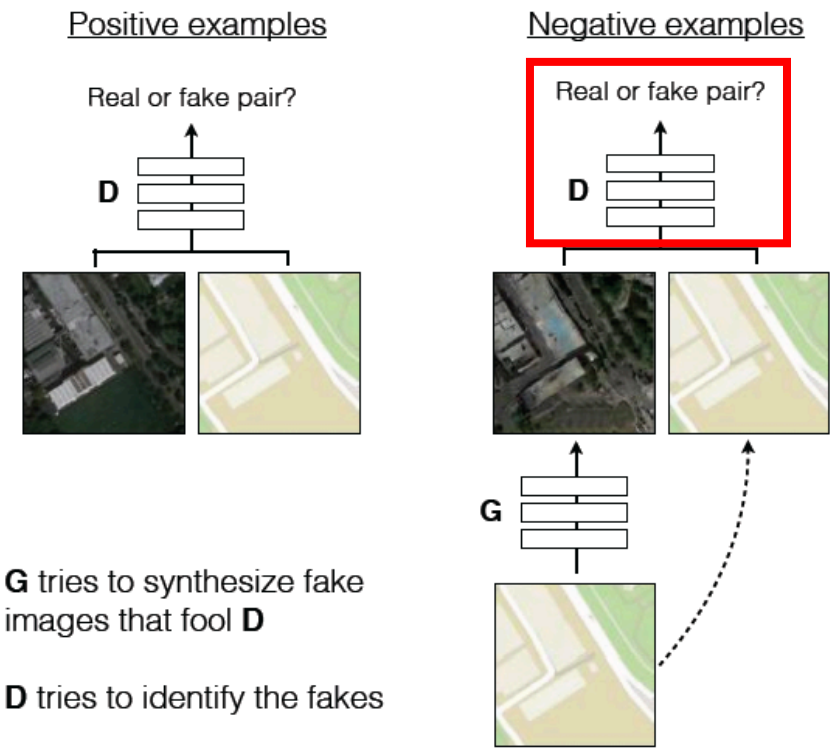
$$G_s^* = \min_{G_s} \mathcal{L}_{\text{content}}(x_c, G_s(x_c)) + \mathcal{L}_{\text{style}}(x_s, G_s(x_c))$$



* Image from Fast Neural Style (ECCV 2016)

Introduction: Prior Work & Motivation

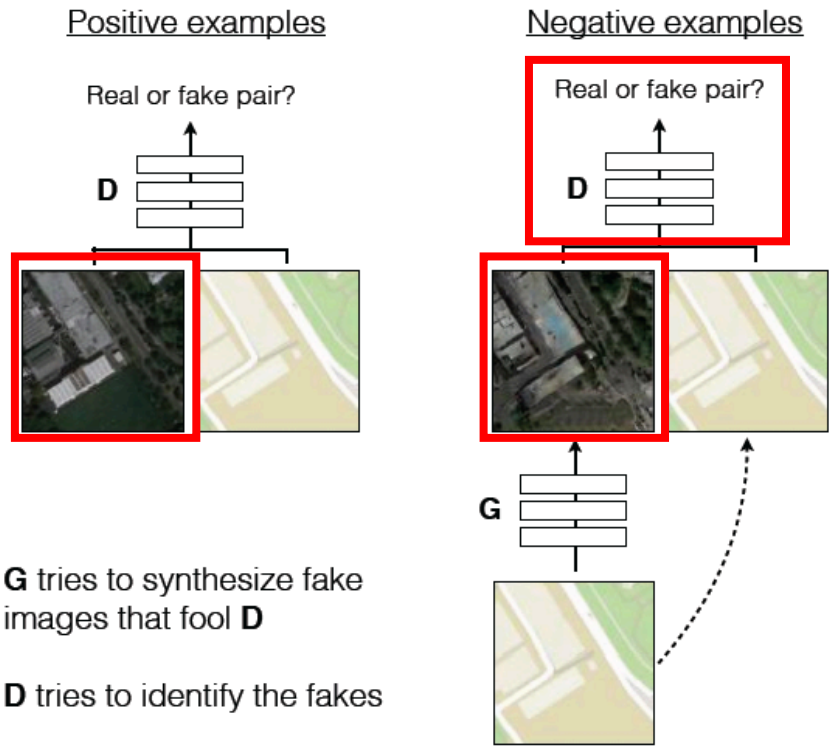
- **Prior work: Pixel-to-pixel (pix2pix)**
  - Train a generator G_{XY} (and a discriminator D_{Y}) for translation $\mathrm{X} \rightarrow \mathrm{Y}$
  - **Domain loss =** GAN loss $(D_{\mathrm{Y}}(y) - 1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x))^2$
  - Better for realistic image generation (previous: *artistic* style transfer)



* Image from pix2pix (CVPR 2017)

Introduction: Prior Work & Motivation

- **Prior work: Pixel-to-pixel (pix2pix)**
  - Train a generator G_{XY} (and a discriminator D_{Y}) for translation $\mathrm{X} \rightarrow \mathrm{Y}$
  - **Domain loss =** GAN loss $(D_{\mathrm{Y}}(y) - 1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x))^2$
  - **Content loss =** L_1 loss (assume paired data (x, y)) $\|G_{\mathrm{XY}}(x) - y\|_1$

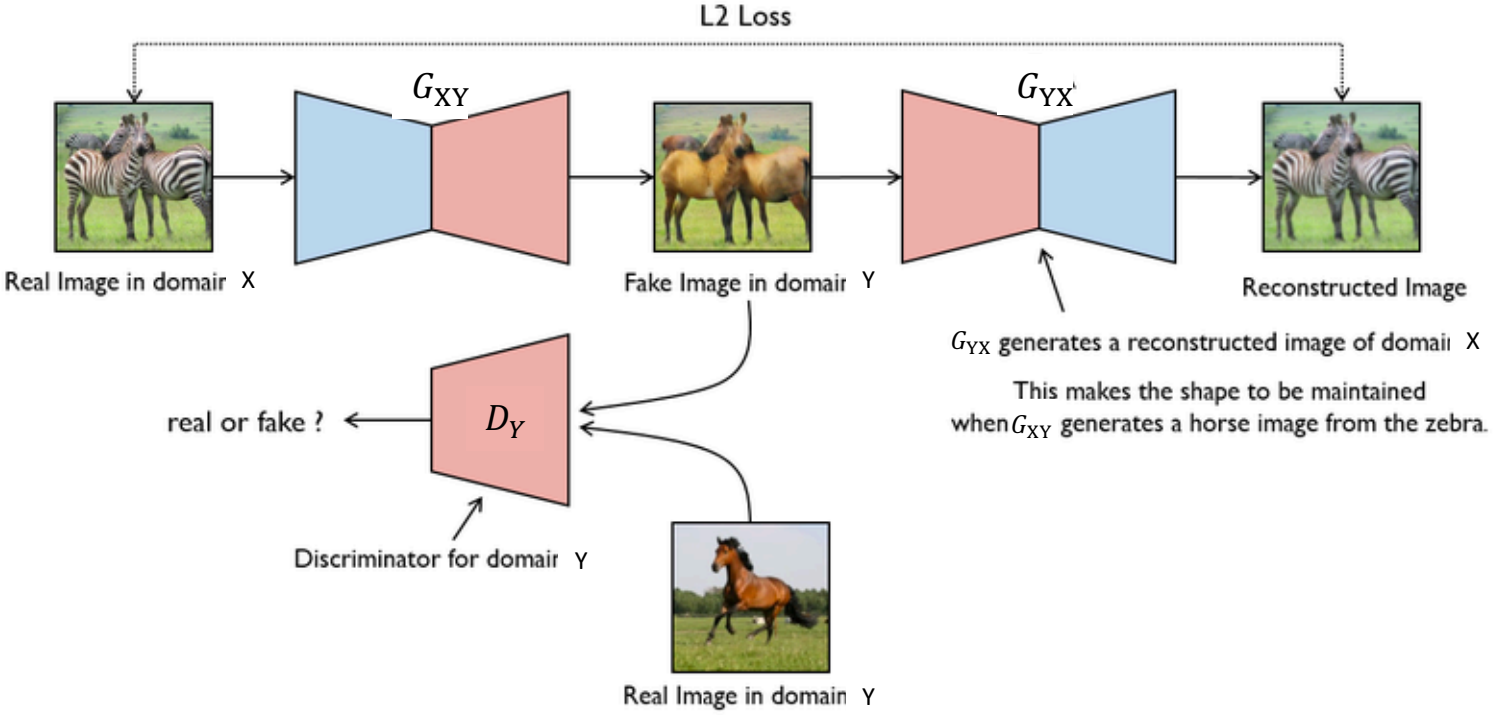


* Image from pix2pix (CVPR 2017)

Introduction: Prior Work & Motivation

- **Prior work: Cycle-consistency GAN (CycleGAN)**
  - Train two generators G_{XY}, G_{YX} (and discriminators D_{X}, D_{Y}) for domain X, Y
  - **Domain loss =** GAN loss $(D_{\mathrm{Y}}(y)-1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x))^2$
  - **Content loss =** Cycle-consistency loss (assume unpaired data x)

$$\|G_{\mathrm{YX}}(G_{\mathrm{XY}}(x)) - x\|_1$$



* Image from CycleGAN (ICCV 2017)

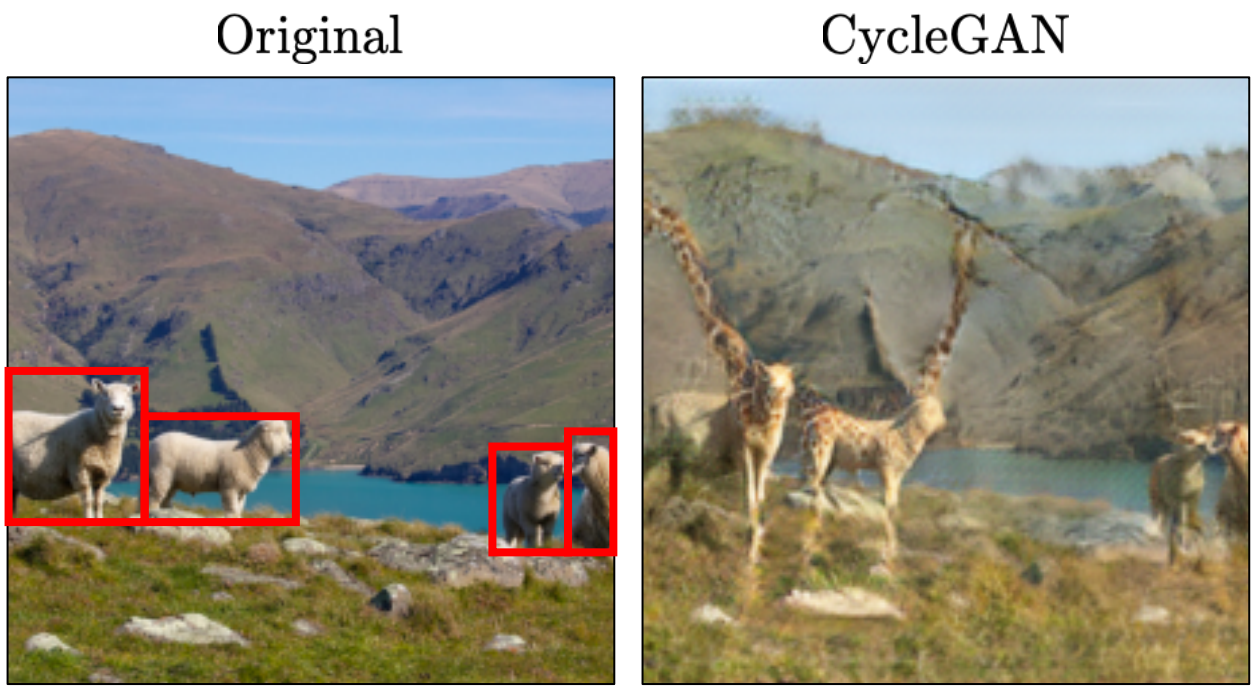
Introduction: Prior Work & Motivation

- **Prior work: Cycle-consistency GAN (CycleGAN)**
  - Translation results on horse→zebra task

* Video from CycleGAN (ICCV 2017)

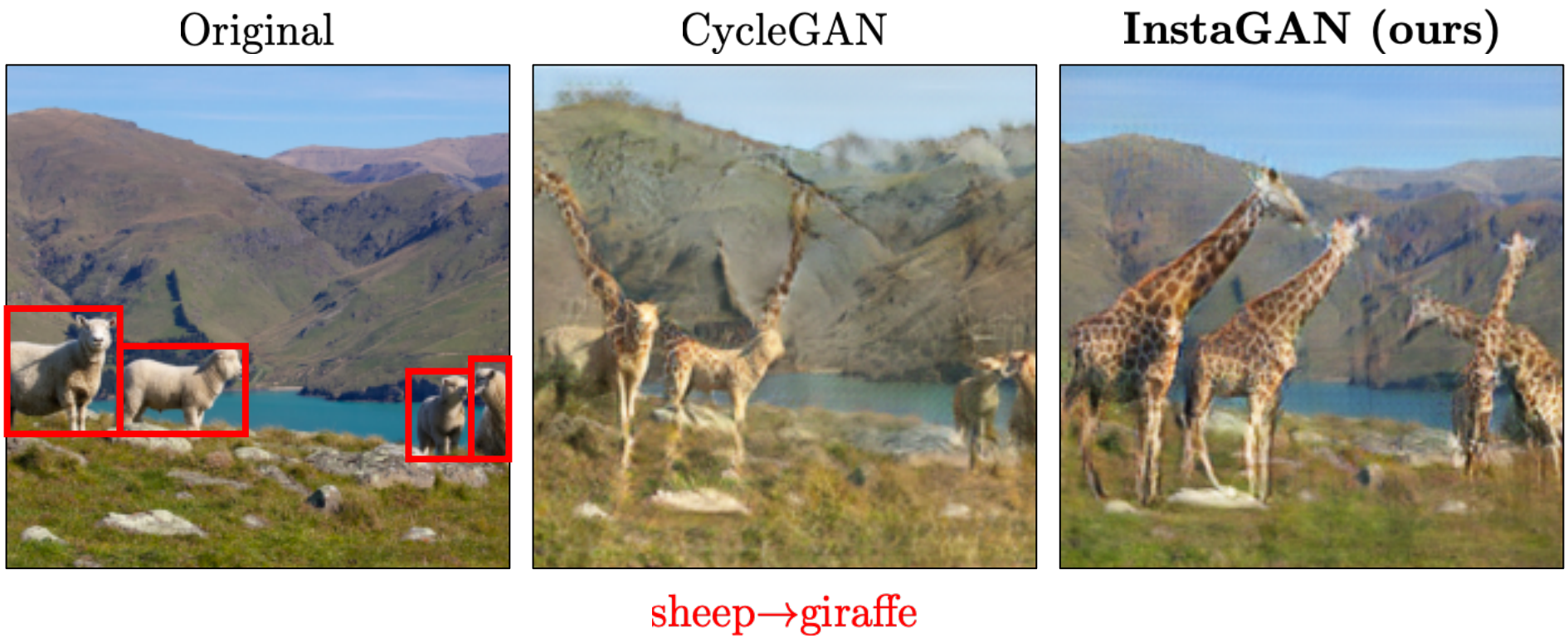
Introduction: Prior Work & Motivation

- However, CycleGAN often fail on challenging tasks:
  1. The translation involves significant changes in the shape of instances
  2. The image contains multiple target instances



Introduction: Our Contribution

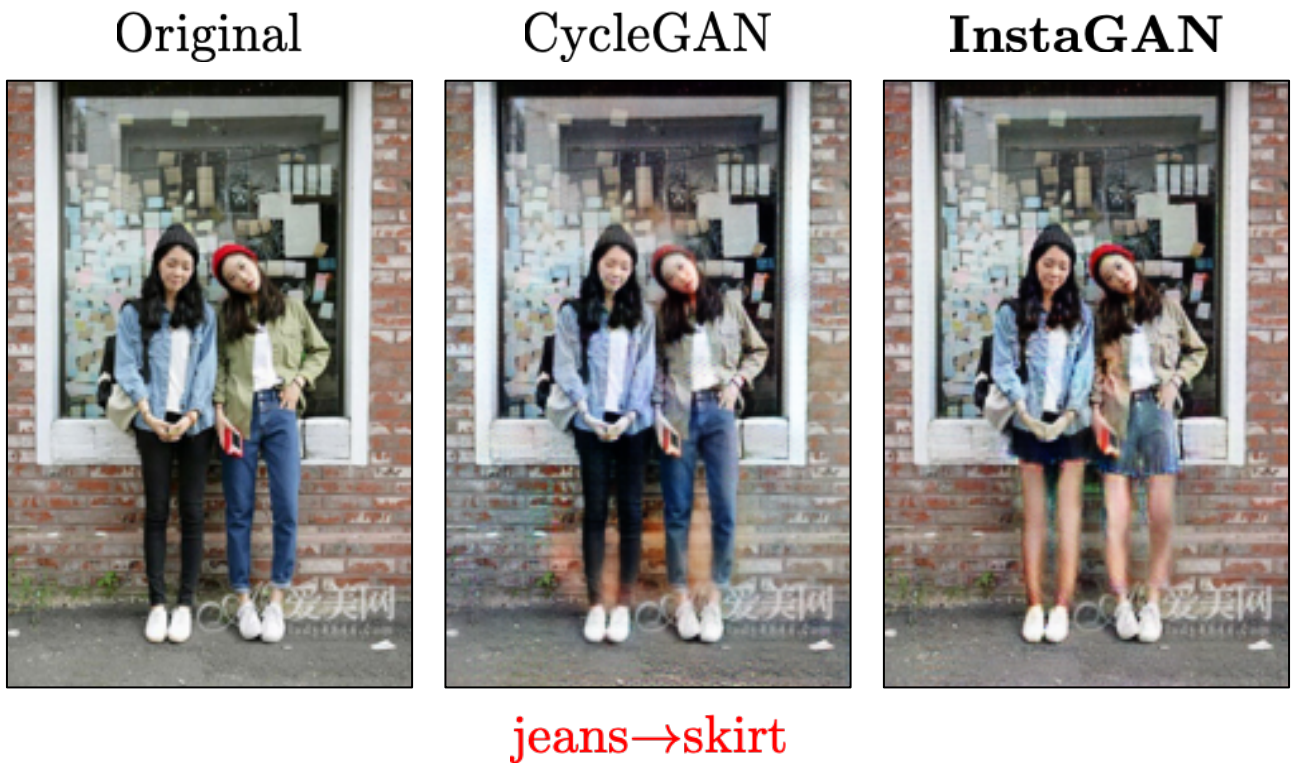
- **InstaGAN (ours)** first reports the results on *multi-instance transfiguration* tasks



Introduction: Our Contribution

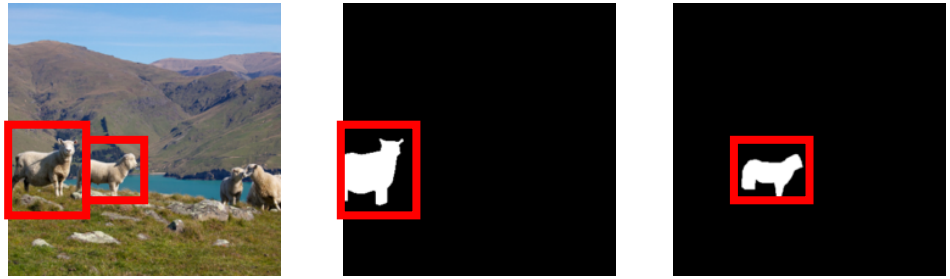
- **InstaGAN (ours)** first reports the results on *multi-instance transfiguration* tasks
  - It can strengthen the applicability of image-to-image translation,

    e.g., *virtual try-on* of fashion items



Our Method: Instance-aware GAN (InstaGAN)

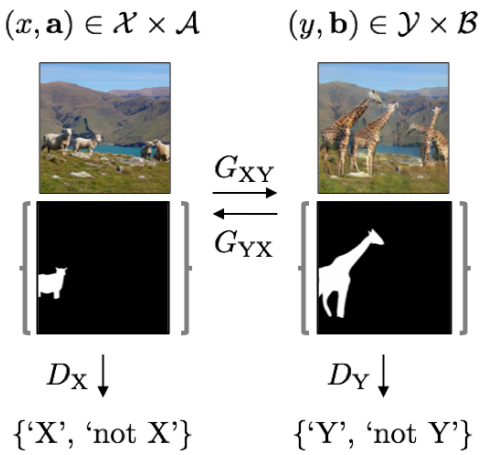
- **Main idea:** Incorporate the additional instance attributes
  - E.g., Segmentation masks are *good representation* of instance shapes



- Learn a joint mapping of an image x and the *set* of instance attributes $\boldsymbol{a} = \{a_i\}_{i=1}^{N}$
  - To this end, we propose

1. Neural architecture
2. Loss function
3. Training/inference scheme

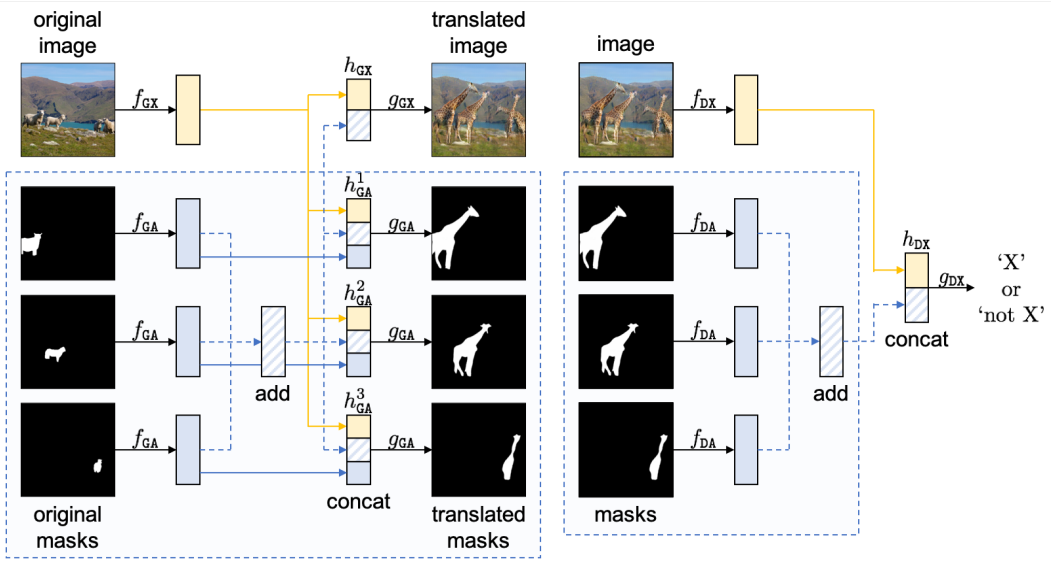
to learn a joint mapping of $(x, \mathbf{a})$



Our Method: InstaGAN Architecture

- **Design architecture** of the generator G and the discriminator D
  - **Issue 1.** It should encode the (arbitrary length) *set* of instance masks
  - **Issue 2.** The results should not be affected by the *order* of instance masks, i.e., image is *permutation invariant*, and mask is *permutation equivariant*

- To achieve the property, we add & concatenate of the features of the set elements



(b) Generator G (c) Discriminator D

Our Method: InstaGAN Architecture

- Computation of the generator G

1. **Extract** the individual features from the *image* and the *masks*

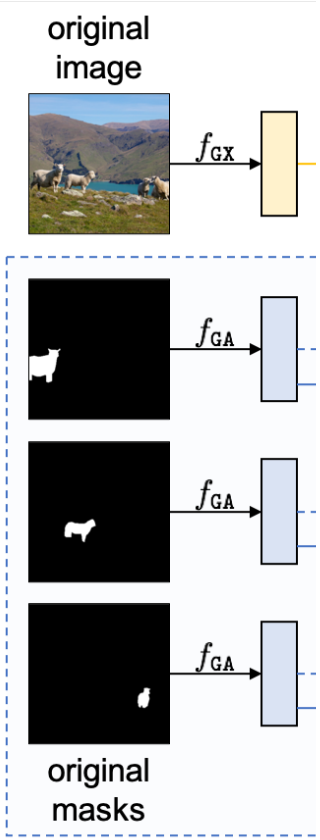
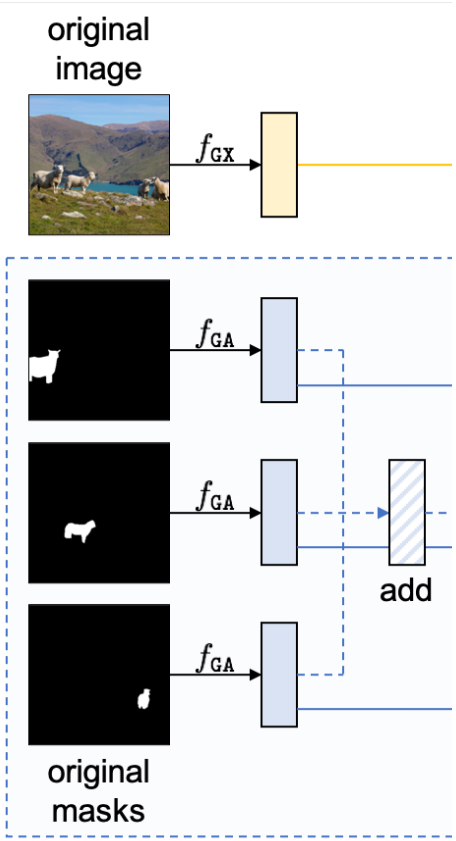


Image encoder f_{GX}
Mask encoder f_{GA}

Our Method: InstaGAN Architecture

- Computation of the generator G

  2. **Add** the *mask features* to obtain the invariant feature (dashed box)

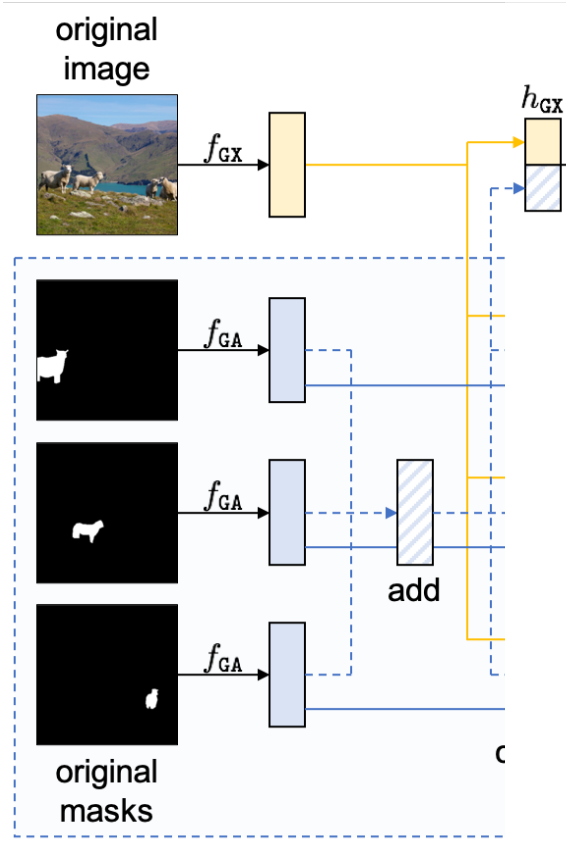


Our Method: InstaGAN Architecture

- Computation of the generator G

(permutation invariant)

3. **Concatenate** the *mask feature* and the *invariant feature* for the **image**

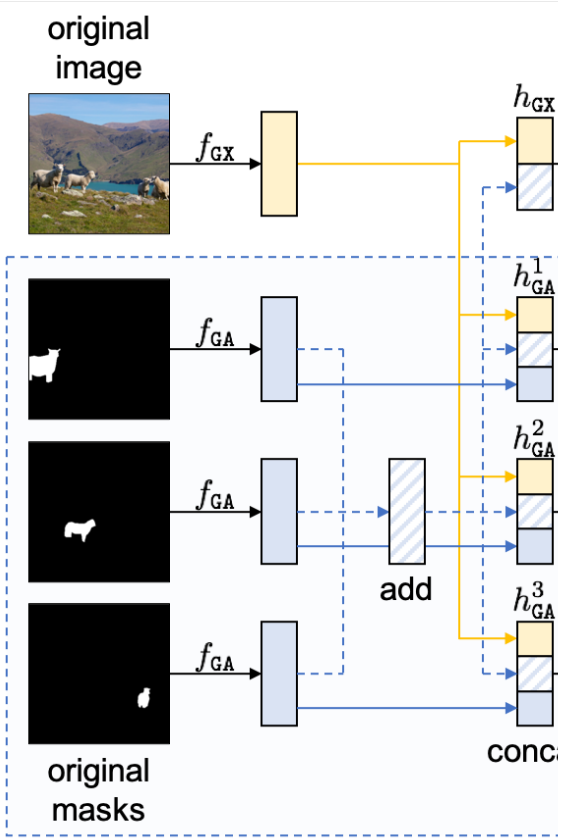


Our Method: InstaGAN Architecture

- Computation of the generator G

(permutation equivariant)

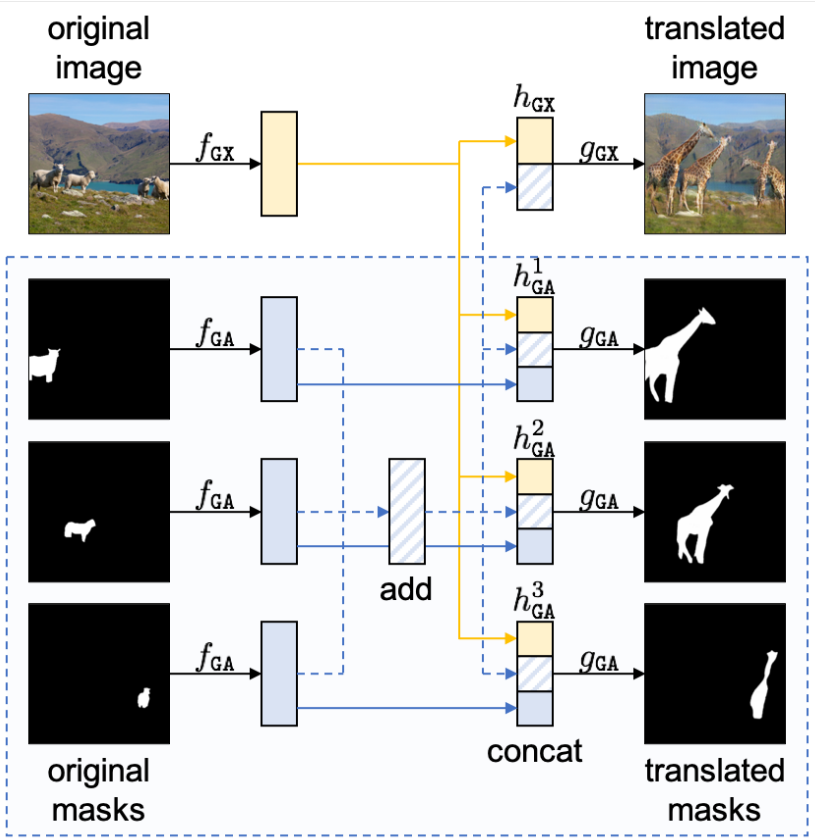
4. **Concatenate** the *image*, *invariant*, and (own) *mask features* for the **mask**



Our Method: InstaGAN Architecture

- Computation of the generator G

  5. **Generate** the translated *image* and *masks*, following the desired property

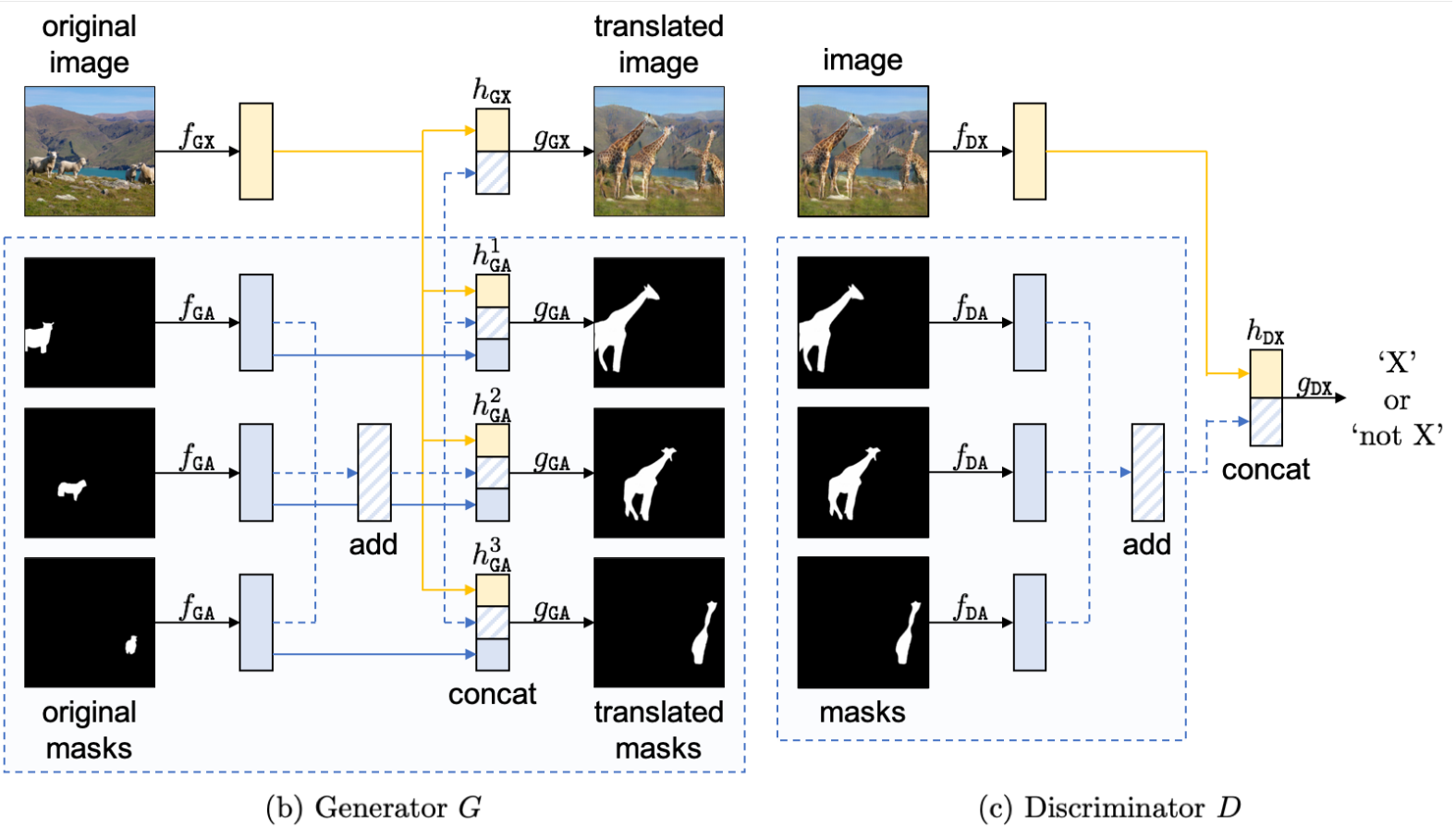


(b) Generator G

Image decoder g_{GX}
Mask decoder g_{GA}

Our Method: InstaGAN Architecture

- Computation of the generator D
  - **Similar** to the image part of the generator G

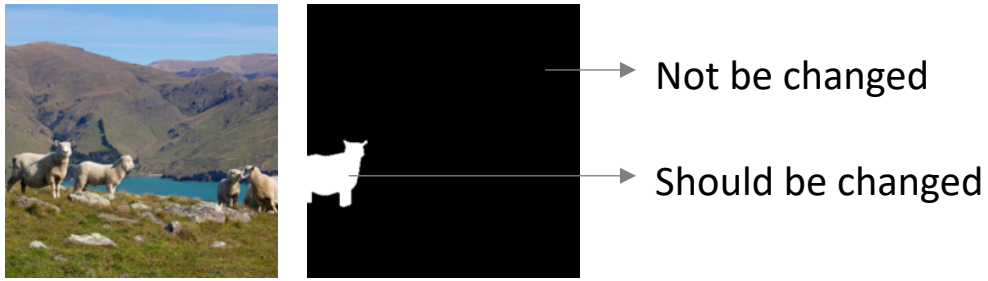


(b) Generator G (c) Discriminator D

Our Method: Context-preserving Loss

- Similar to **CycleGAN**, we use
  - GAN loss for **domain** loss $(D_{\mathrm{Y}}(y, \mathbf{b}) - 1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x, \mathbf{a}))^2$
  - Cycle-consistency loss for **content** loss $\|G_{\mathrm{YX}}(G_{\mathrm{XY}}(x, \mathbf{a})) - (x, \mathbf{a})\|_1$

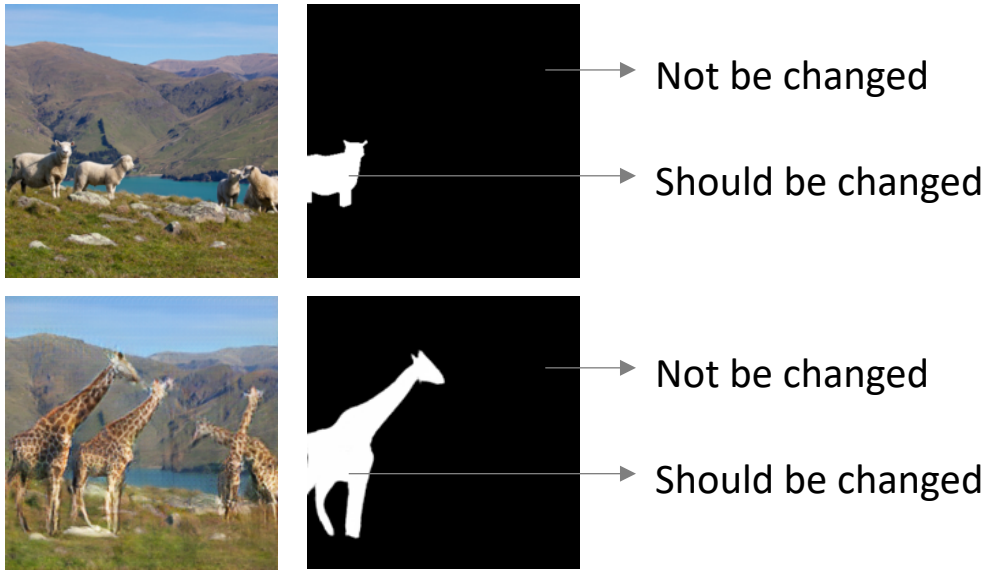
- In addition, we propose a new content loss using instance masks
  - Background (outside of instance masks) should **not** be changed



Our Method: Context-preserving Loss

- Similar to **CycleGAN**, we use
  - GAN loss for **domain** loss $(D_{\mathrm{Y}}(y, \mathbf{b}) - 1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x, \mathbf{a}))^2$
  - Cycle-consistency loss for **content** loss $\|G_{\mathrm{YX}}(G_{\mathrm{XY}}(x, \mathbf{a})) - (x, \mathbf{a})\|_1$

- In addition, we propose a new content loss using instance masks
  - Also, we should consider **both** *original* and *translated* instance shapes



Our Method: Context-preserving Loss

- Similar to **CycleGAN**, we use
  - GAN loss for **domain** loss $(D_{\mathrm{Y}}(y, \mathbf{b}) - 1)^2 + D_{\mathrm{Y}}(G_{\mathrm{XY}}(x, \mathbf{a}))^2$
  - Cycle-consistency loss for **content** loss $\|G_{\mathrm{YX}}(G_{\mathrm{XY}}(x, \mathbf{a})) - (x, \mathbf{a})\|_1$

- In addition, we propose a new content loss using instance masks
  - For $(y', \mathbf{b}') = G_{\mathrm{XY}}(x, \mathbf{a})$, the **context-preserving loss** is given by

$$\|w(\mathbf{a}, \mathbf{b}') \odot (x - y')\|_1$$

  where $w(\mathbf{a}, \mathbf{b}')$ is (one minus) the union of two masks $\mathbf{a}$ and $\mathbf{b}'$

- The **total loss** of InstaGAN is given by

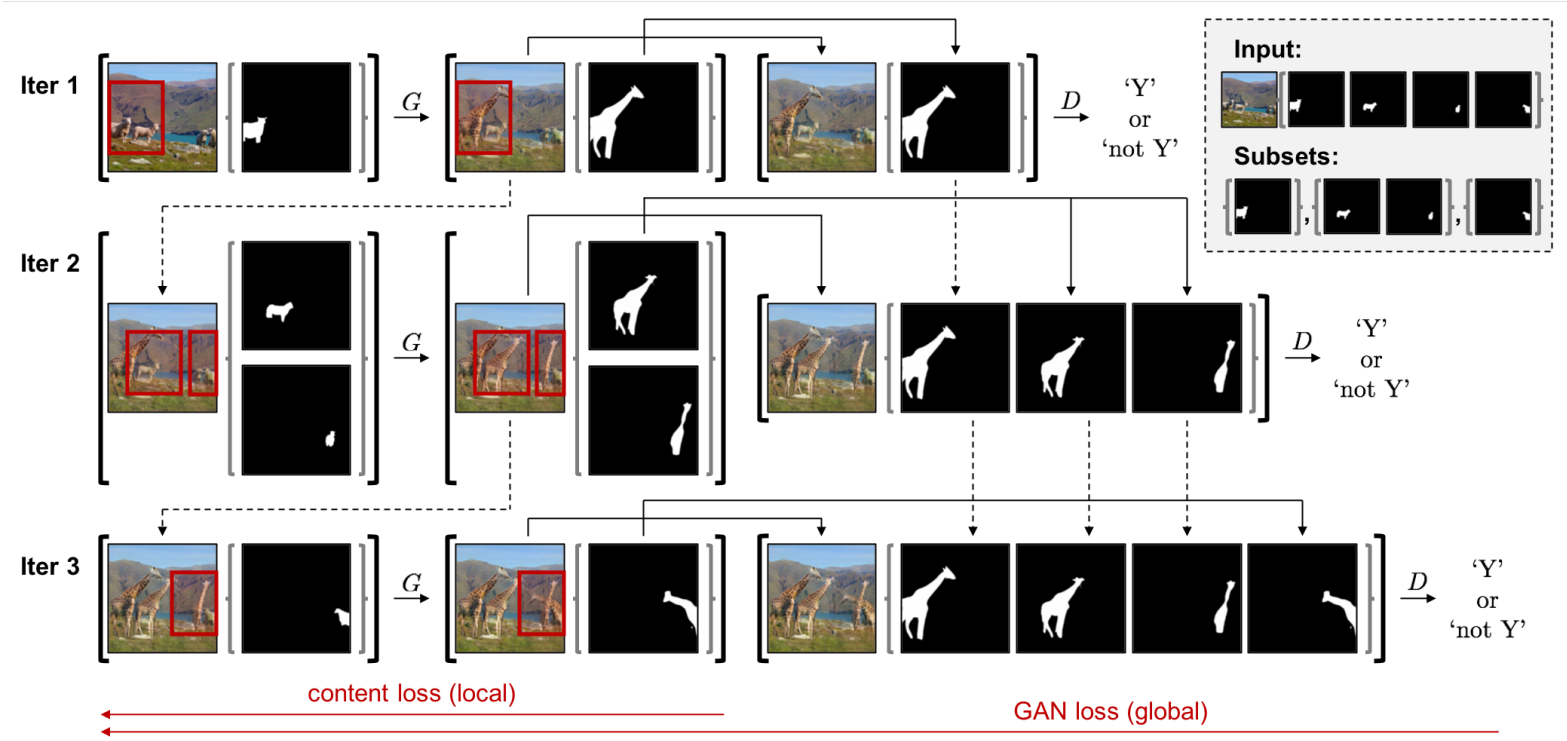
$$\mathcal{L}_{\mathrm{InstaGAN}} = \mathcal{L}_{\mathrm{GAN}}((x, \mathbf{a}), (y', \mathbf{b}')) + \mathcal{L}_{\mathrm{content}}((x, \mathbf{a}), (y', \mathbf{b}'))$$

Our Method: Sequential Mini-batch Translation

- **Motivation:** GPU memory linearly increases to the *number of training instances*

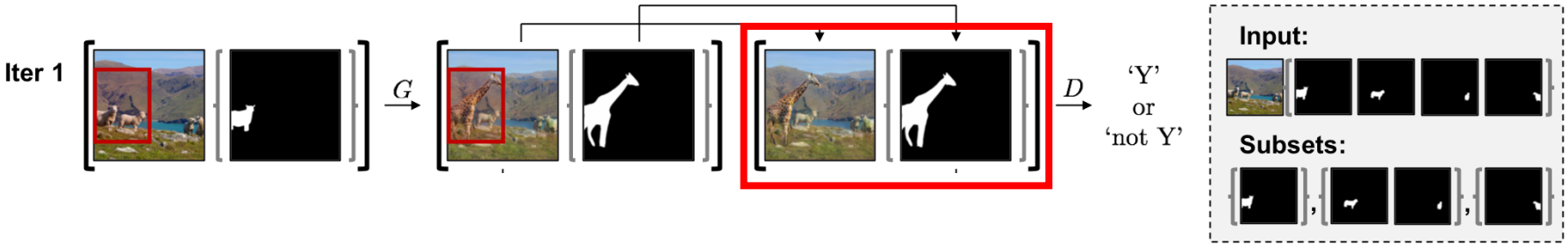
- **Idea:** Divide instances into the mini-batches and sequentially translate them

  ⇒ Only need the GPU memory of the size of the mini-batch (fixed)



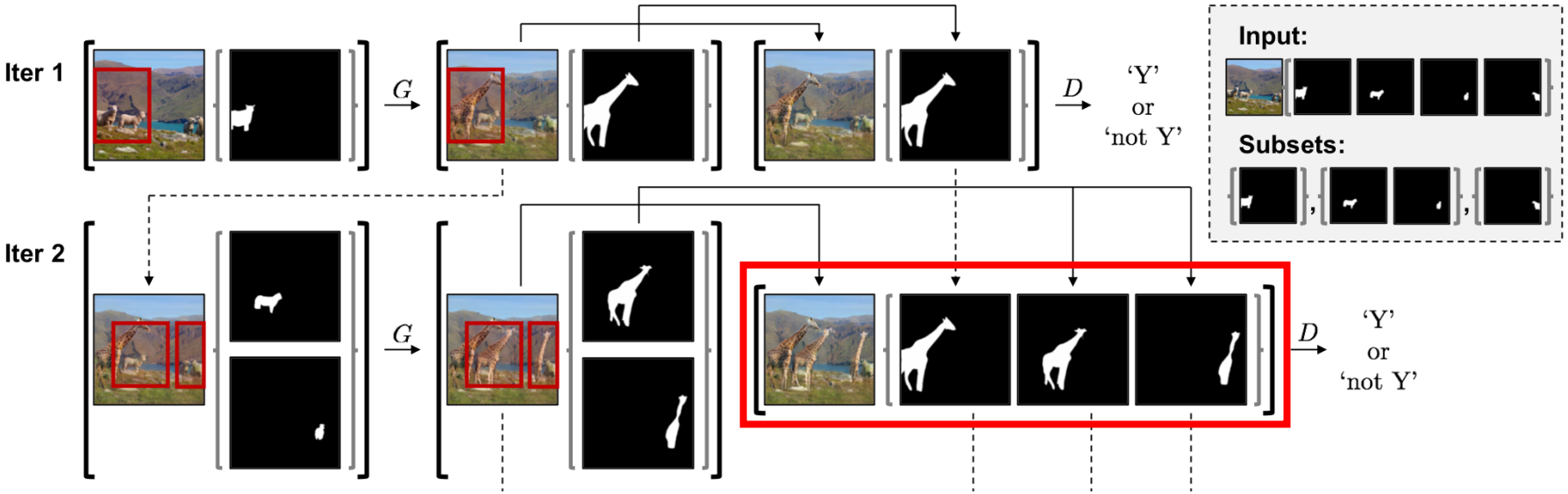
Our Method: Sequential Mini-batch Translation

- **Sequential inference:**
  - Sequentially translate the mini-batch of the instances
  - Aggregate all the translated masks to align with the image



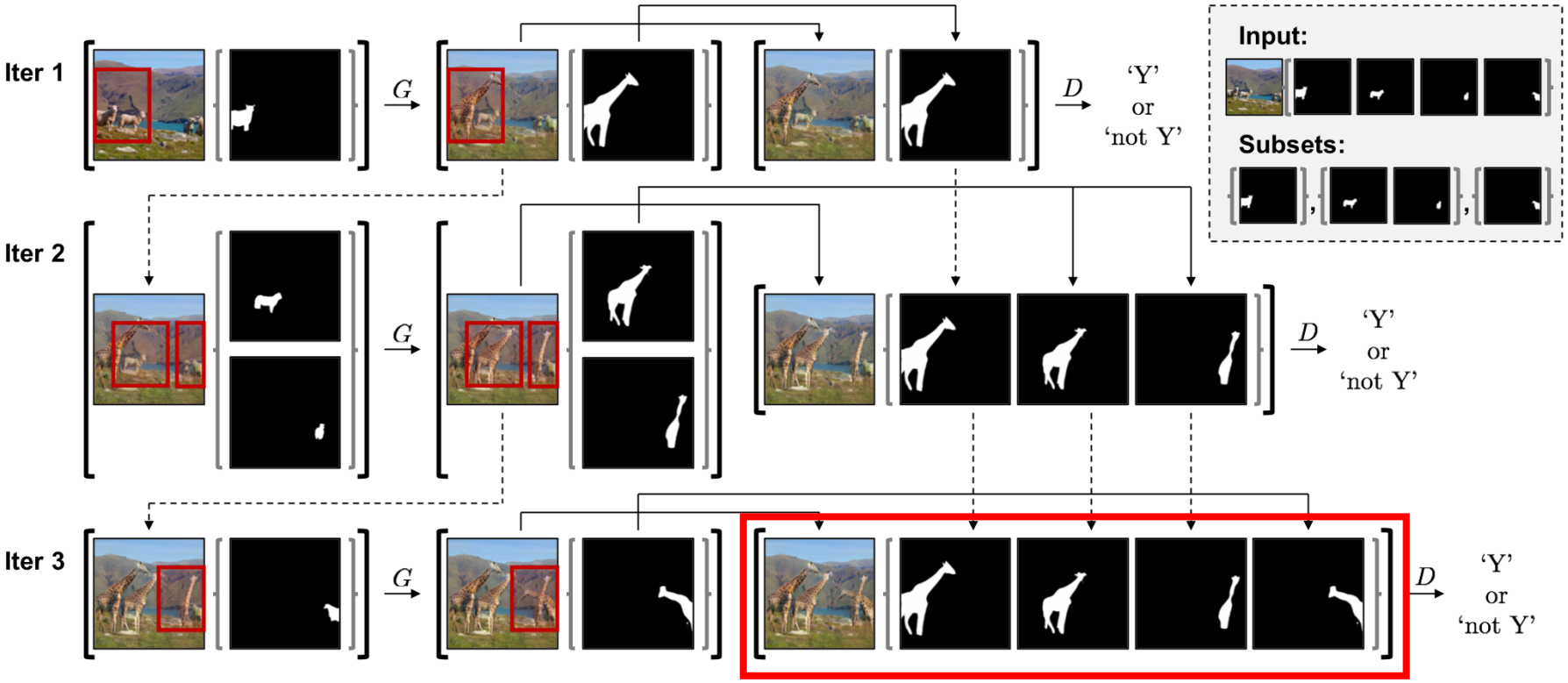
Our Method: Sequential Mini-batch Translation

- **Sequential inference:**
  - Sequentially translate the mini-batch of the instances
  - Aggregate all the translated masks to align with the image



Our Method: Sequential Mini-batch Translation

- **Sequential inference:**
  - Sequentially translate the mini-batch of the instances
  - Aggregate all the translated masks to align with the image



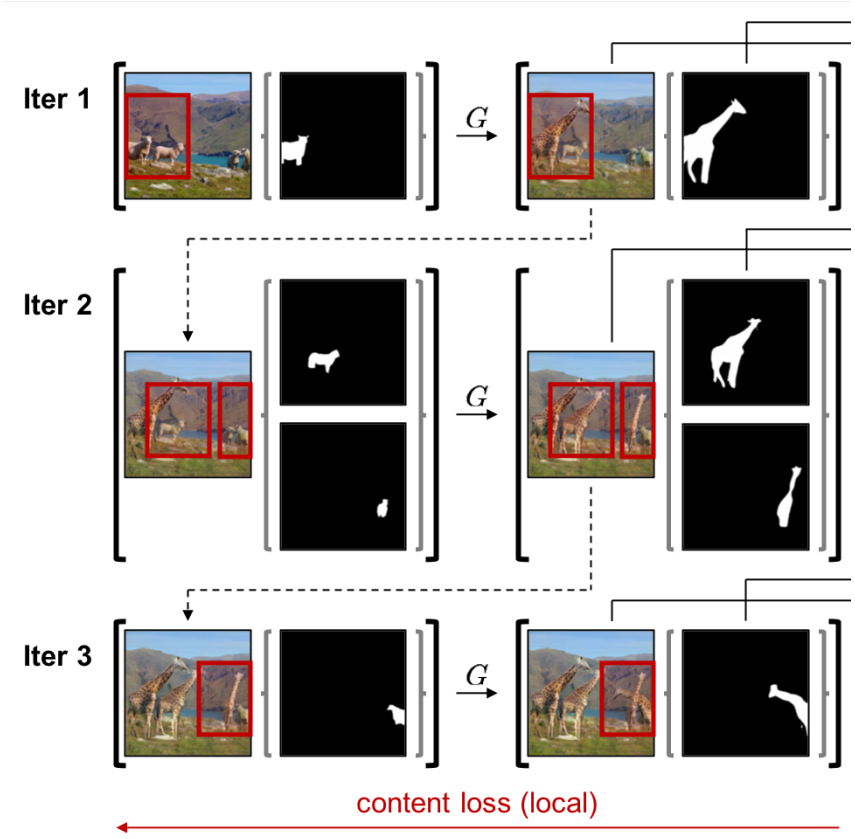
Our Method: Sequential Mini-batch Translation

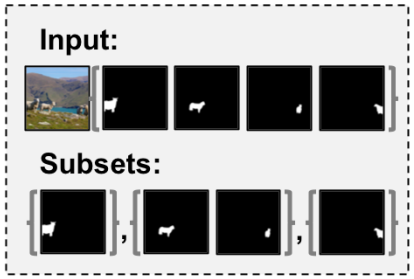
- **Sequential training:**
  - We also propose a training algorithm corresponding to the inference scheme

Our Method: Sequential Mini-batch Translation

- **Sequential training:**
  - Apply content loss to the intermediate samples of current mini-batch

    (as the content loss is given by the *local results* of the generator)

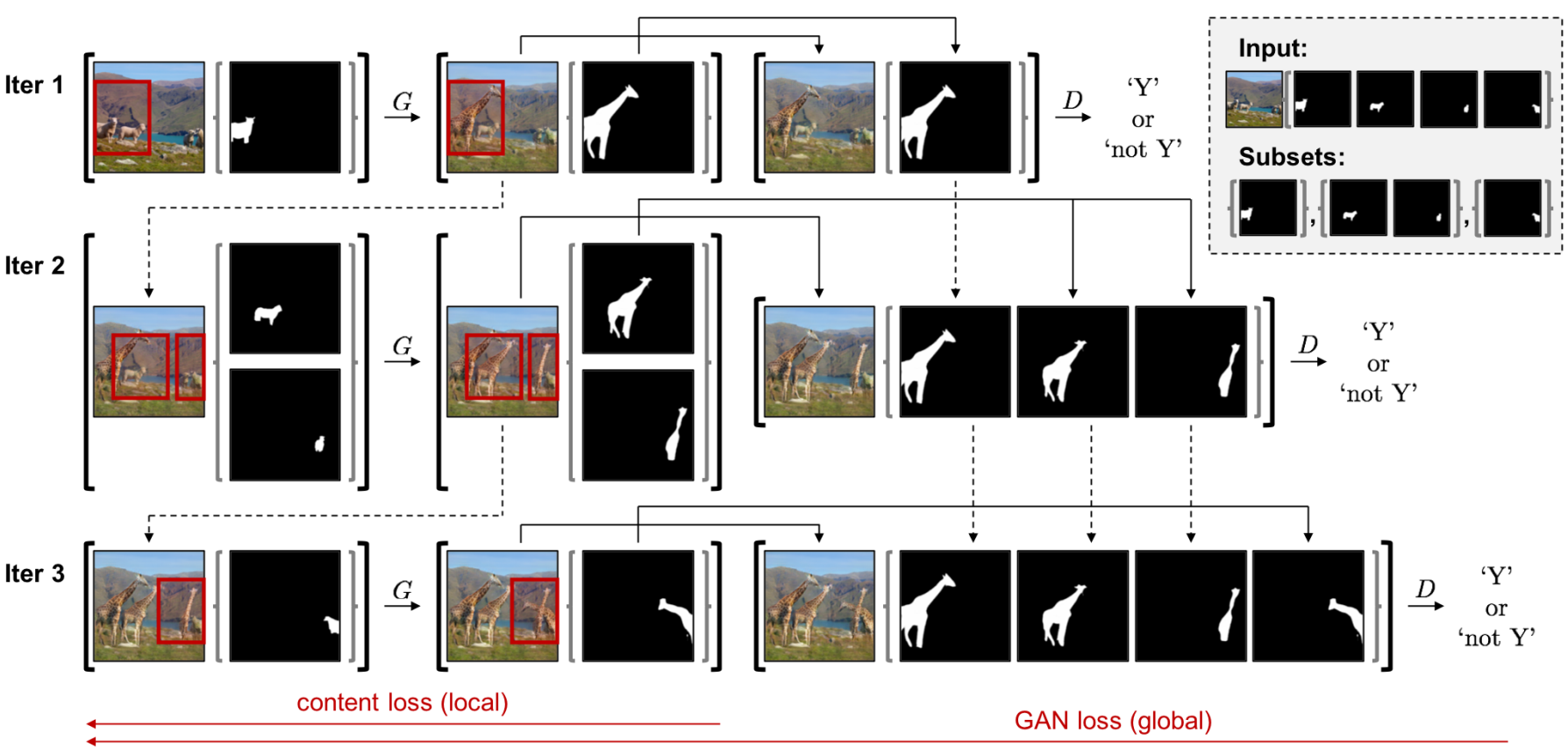




Our Method: Sequential Mini-batch Translation

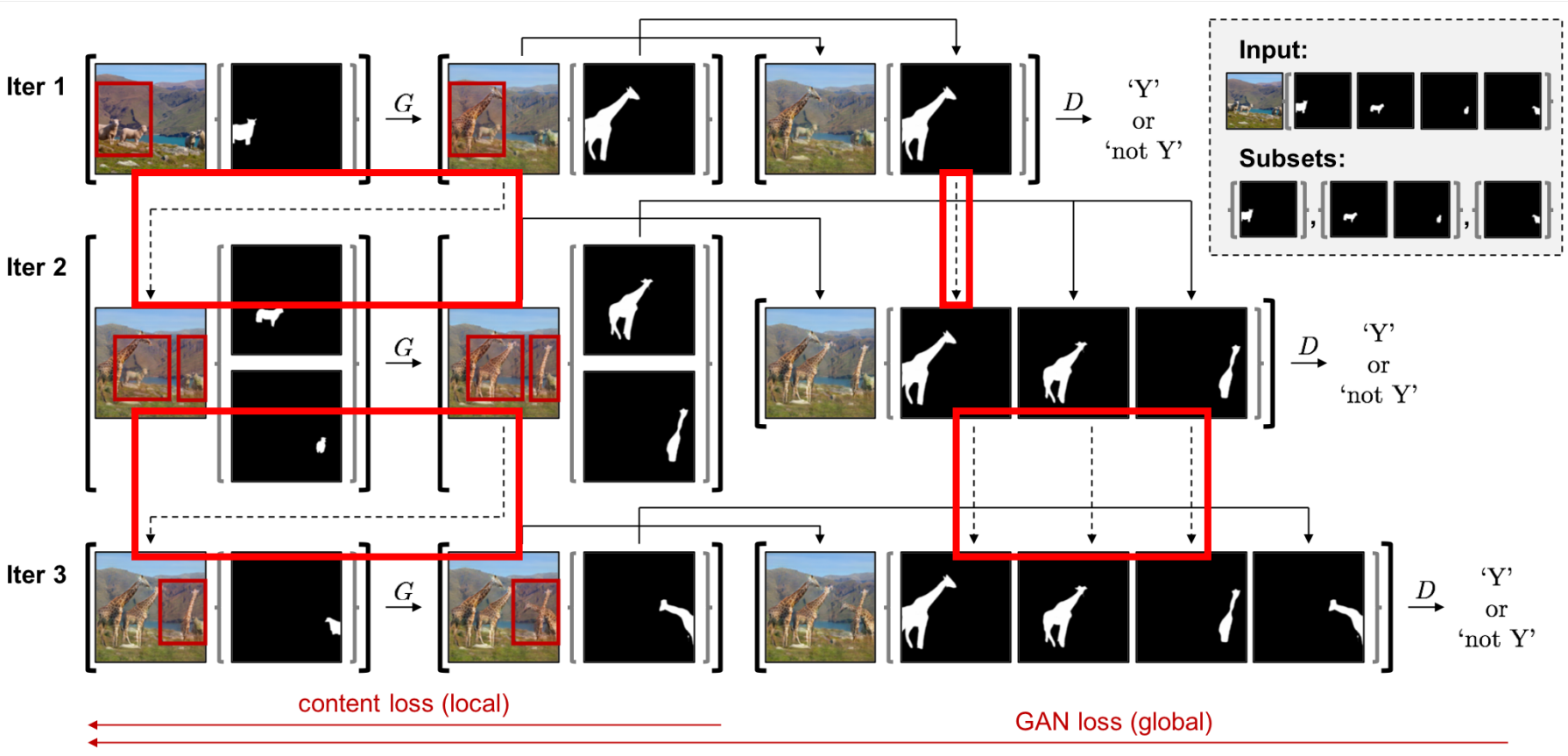
- **Sequential training:**
  - Apply domain (GAN) loss to the samples of aggregated mini-batches

    (as the original image $\{x\}$ contains the *full set* of instance masks)

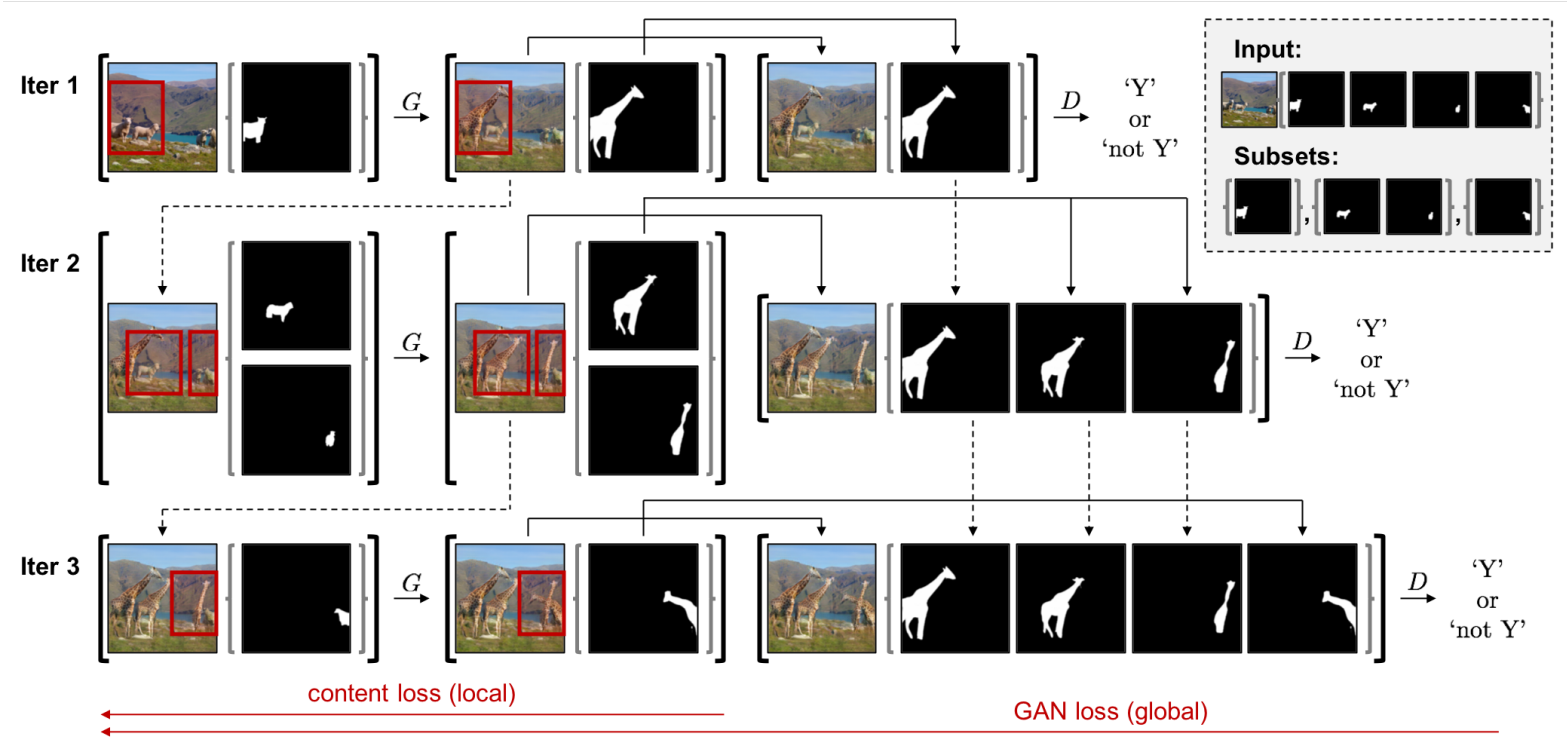


Our Method: Sequential Mini-batch Translation

- **Sequential training:**
  - Detach every m-th iteration of training (dashed line), so that only a fixed GPU memory is required, regardless of the number of training instances



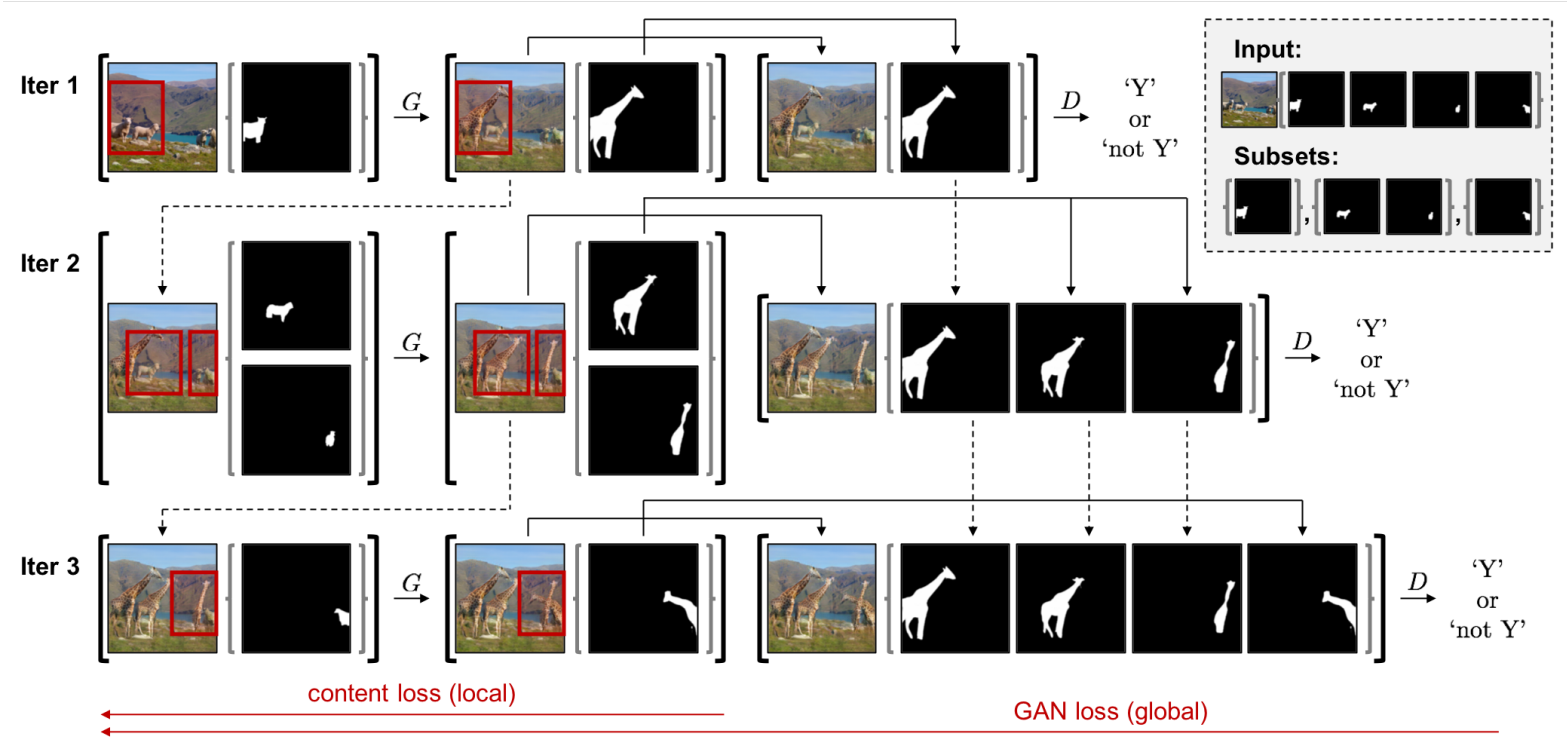
Our Method: Sequential Mini-batch Translation



- The **sequential version** of the loss of the InstaGAN is given by

$$\mathcal{L}_{\text{InstaGAN-SM}} = \sum_{m=1}^{M} \mathcal{L}_{\text{GAN}}((x, \mathbf{a}), (y'_m, \mathbf{b}'_{1:m})) + \mathcal{L}_{\text{content}}((x_m, \mathbf{a}_m), (y'_m, \mathbf{b}'_m))$$

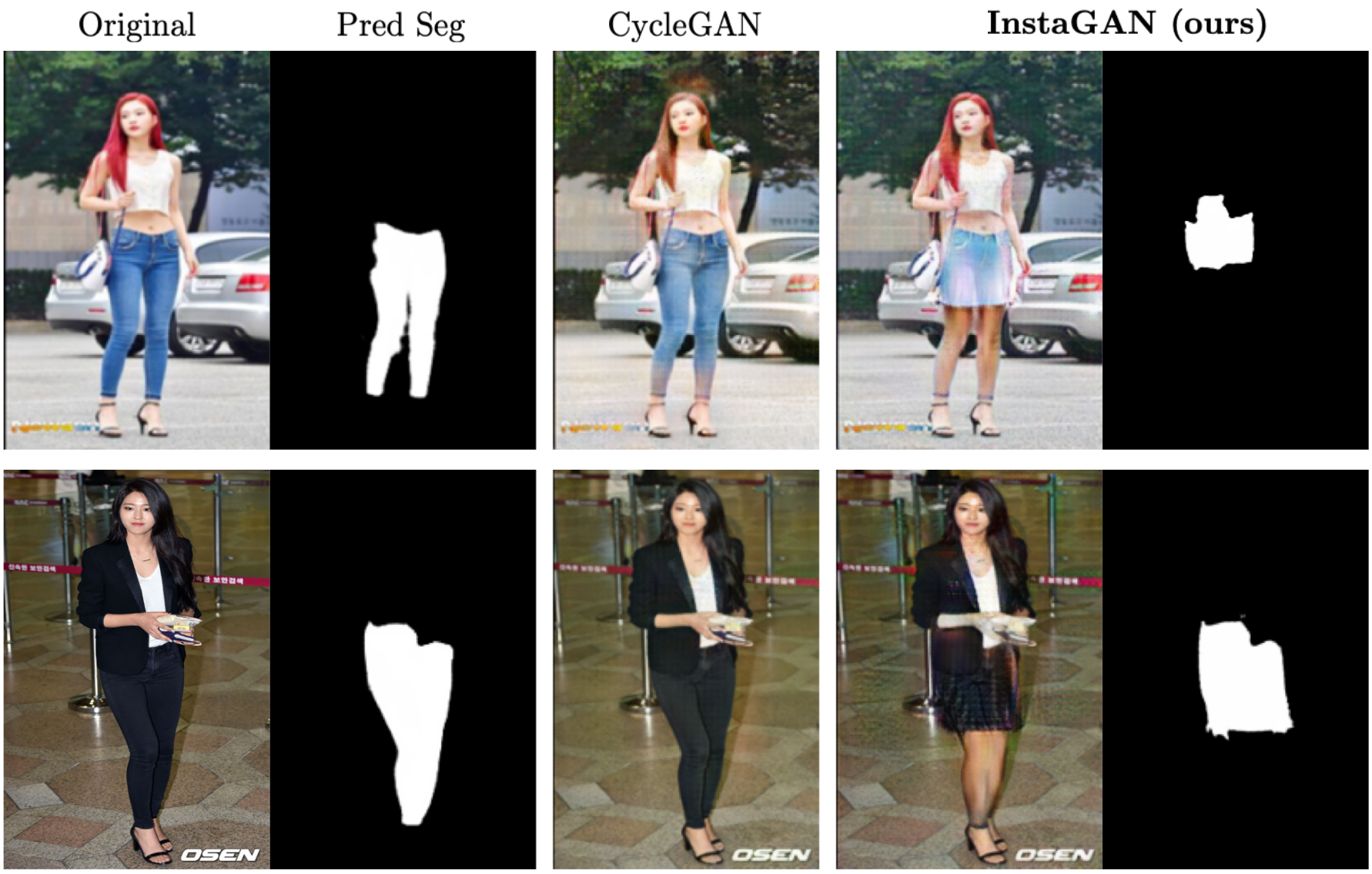
Our Method: Sequential Mini-batch Translation



- Sequential inference/training
  1. Improves **many-instance** samples by allowing the training of them
  2. Also improves **few-instance** samples due to its data augmentation effect

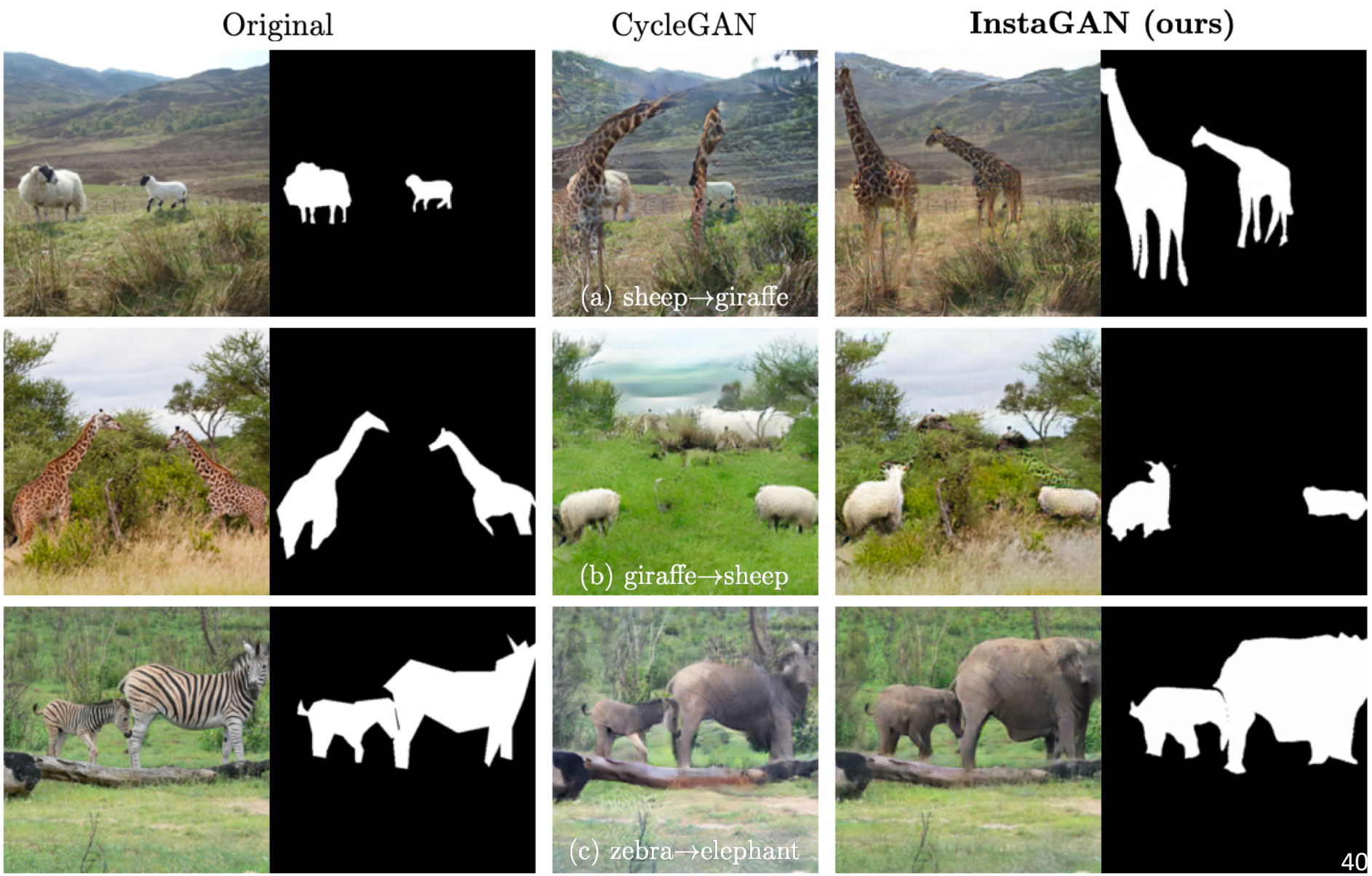
Experimental Results: Fashion Dataset

Experimental Results: Fashion Dataset

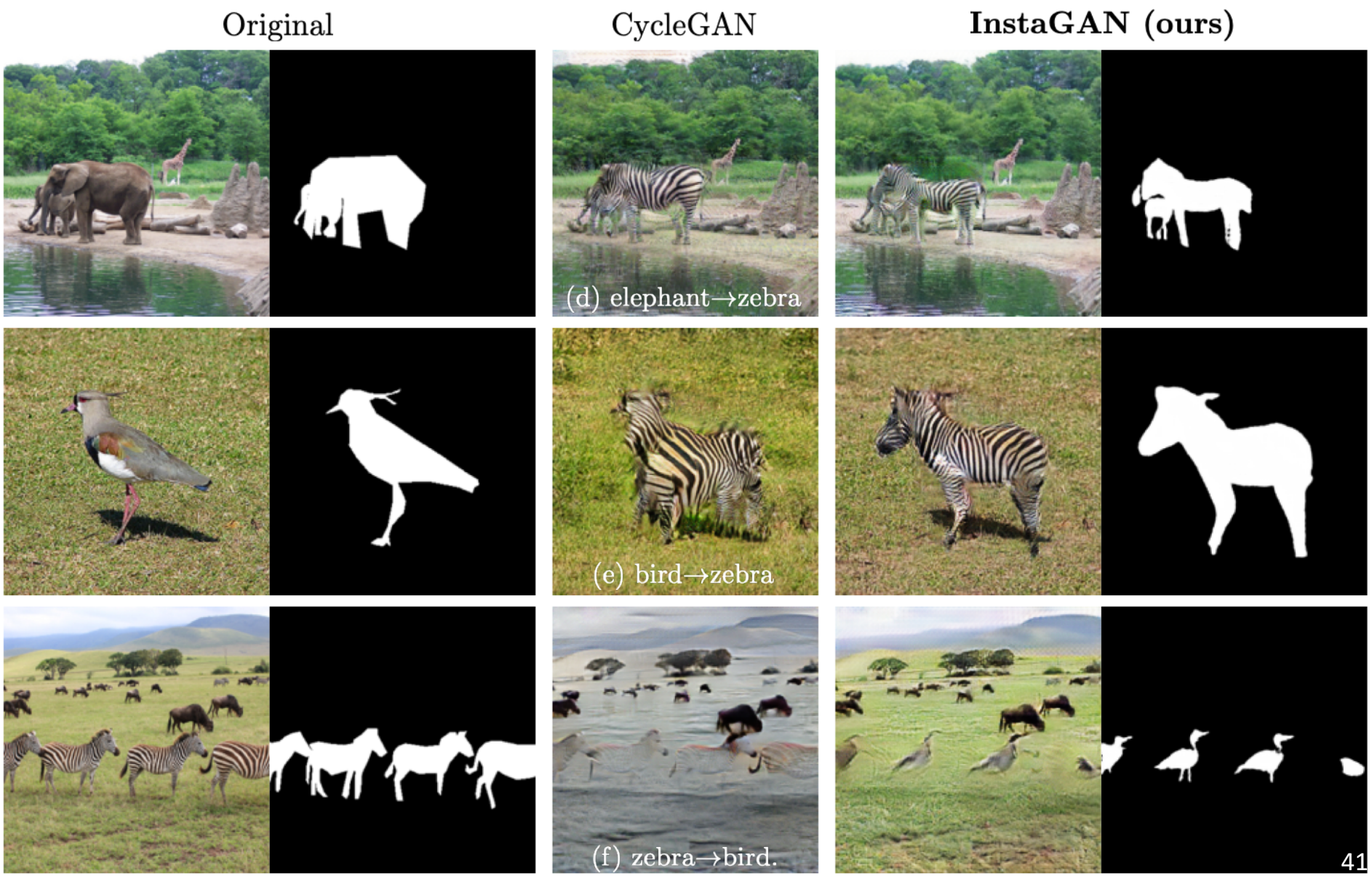


(c) pants→skirt with predicted masks

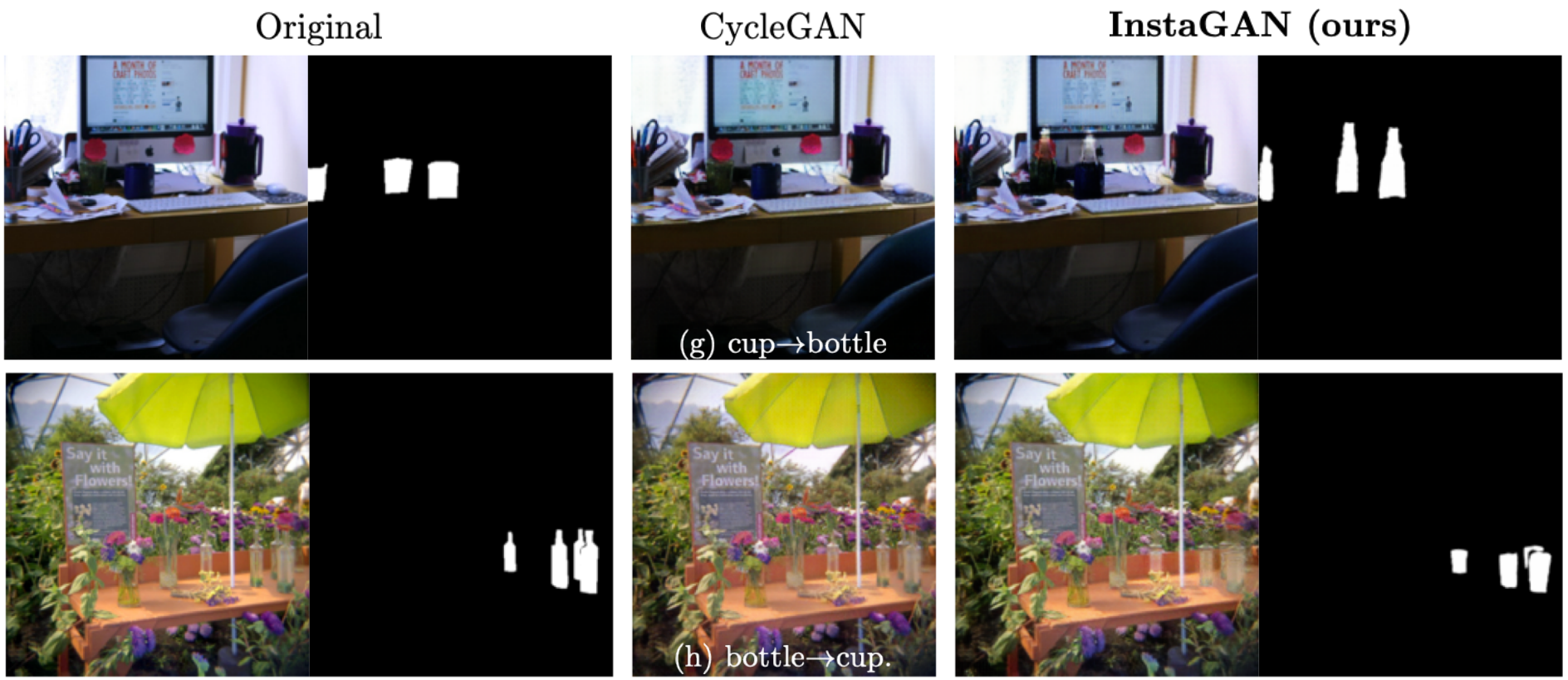
Experimental Results: COCO Dataset



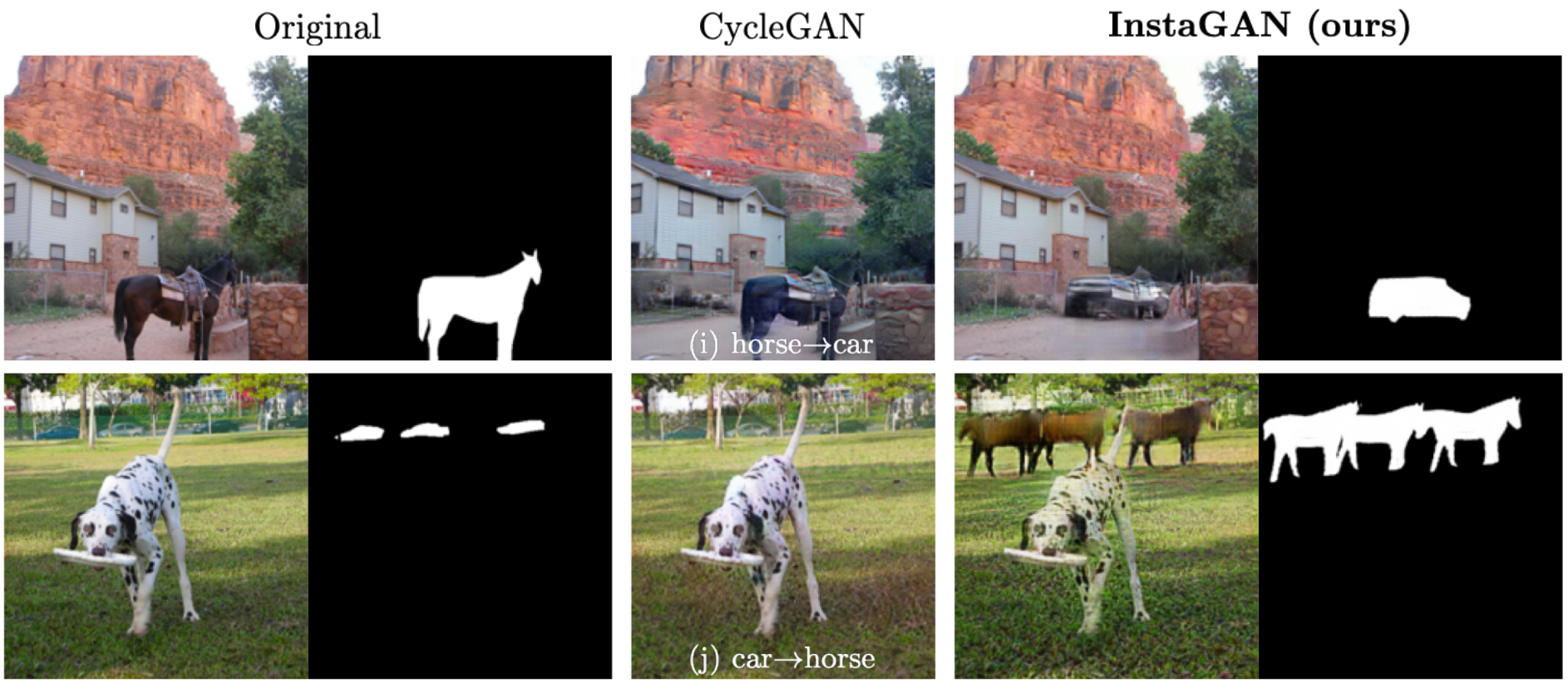
Experimental Results: COCO Dataset



Experimental Results: COCO Dataset



Experimental Results: COCO Dataset



Experimental Results: More Translation

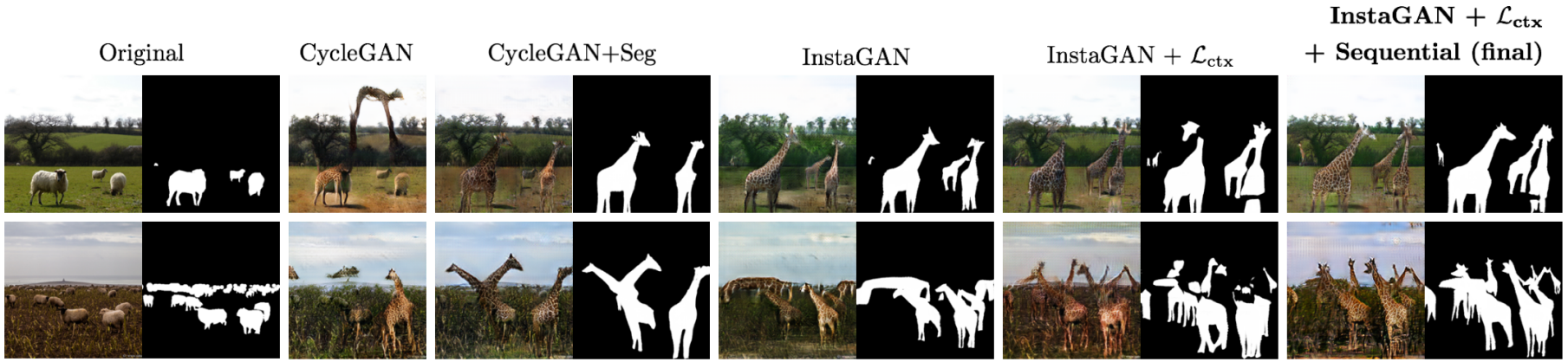
- Our method can control the instances to translate by conditioning input

- Our method smoothly adopt the target instances over epoch increases

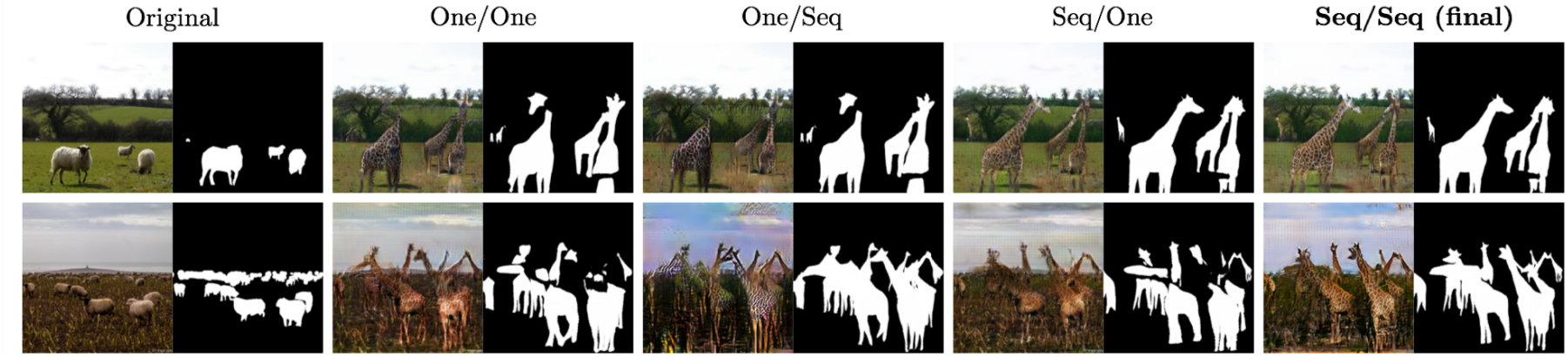
- Our method produces temporally coherent results on video translation

Experimental Results: Ablation Study

- Each component of our method (architecture, loss, algorithm) are essential



- Both sequential training (left) and inference (right) are essential



Experimental Results: Quantitative Results

- Our method show better classification score[1] than the baseline model (CycleGAN)

Table 1: Classification score for CCP dataset.

	jeans→skirt		skirt→jeans		shorts→pants		pants→shorts	
	train	test	train	test	train	test	train	test
Real	0.970	0.888	0.982	0.946	1.000	0.984	0.990	0.720
CycleGAN	0.465	0.371	0.561	0.483	0.845	0.524	0.305	0.085
InstaGAN (ours)	**0.665**	**0.600**	**0.658**	**0.540**	**0.898**	**0.768**	**0.373**	**0.232**

Table 2: Classification score (masked) for COCO dataset.

	sheep→giraffe		giraffe→sheep		cup→bottle		bottle→cup	
	train	test	train	test	train	test	train	test
Real	0.891	0.911	0.925	0.930	0.746	0.723	0.622	0.566
CycleGAN	0.313	0.594	0.291	0.512	0.368	0.403	0.290	0.275
InstaGAN (ours)	**0.406**	**0.781**	**0.355**	**0.642**	**0.443**	**0.465**	**0.322**	**0.333**

1. Ratio of the pre-trained classifier classifies the translated image as the target domain

Thank you for your attention !